

Faculty of Media Engineering and Technology
German University in Cairo



LLM Hallucination Detection via Knowledge Augmentation

Bachelor Thesis

Author: Mohammed Mahmoud Elnaggar

Supervisors: Dr. Nourhan Ehab

Submission Date: 7 June, 2026

This is to certify that:

- (i) the thesis comprises only my original work toward the Bachelor Degree
- (ii) due acknowledgement has been made in the text to all other material used

Mohammed Mahmoud Elnaggar
7 June, 2026

Acknowledgments

I would like to express my sincere gratitude to my supervisor, Dr. Nourhan Ehab, for her invaluable guidance and continuous encouragement throughout the course of this thesis. Her thoughtful feedback and unwavering support were instrumental in shaping this work from its early stages to its final form.

I am also deeply grateful to the Media Engineering and Technology Faculty at the German University in Cairo for providing the academic environment and resources that made this research possible.

Finally, and most importantly, I would like to thank my family for their endless emotional support and encouragement. Their belief in me throughout this journey gave me the strength to see it through.

Abstract

Large language models (LLMs) produce highly fluent responses across a wide range of tasks. However, LLMs tend to hallucinate and produce factually and logically inconsistent outputs, which makes them unreliable in fields that require accurate information. Existing approaches to detect hallucinations fall into several categories, the two main ones being zero-resource and external knowledge-based approaches. Zero-resource approaches detect hallucinations through response consistency checking without consulting any external source. These approaches have a fundamental limitation: if an LLM consistently produces the same hallucination across multiple samples of the same prompt, the system fails to flag it. Furthermore, zero-resource approaches cannot verify claims that fall outside the LLM’s parametric knowledge. Most external knowledge-based approaches using structured representations still rely on Natural Language Inference (NLI) models for triple comparison against the knowledge source, rather than performing direct symbolic matching. This reliance on natural language inference reintroduces the ambiguity that structured knowledge representation was intended to eliminate. Additionally, most existing approaches do not focus on logical inconsistency detection or provide fine-grained categorization of the hallucinations produced.

In this work, we propose KGVS (Knowledge Graph Verification System), an external knowledge-based hallucination detection system that verifies LLM outputs against data stored in a knowledge graph. KGVS operates in a closed domain where the domain rules and facts are stored in a knowledge graph. The system extracts triples from the LLM output and checks them for logical inconsistencies by comparing triples against each other, and for factual inconsistencies by comparing them against evidence retrieved from the knowledge graph. The system also categorizes detected hallucinations as either factually or logically inconsistent, with further fine-grained subcategories for each type. KGVS was evaluated across four LLMs on a custom dataset of 100 domain-specific questions, collectively generating 1,819 triples. The system achieved an overall F1 score of 91.75% with a detection rate of 97.32%, showing that, within a closed-domain setting, direct triple-to-triple comparison against a structured knowledge base can effectively address the core weakness of consistency-based approaches — their inability to detect hallucinations that a model produces consistently across multiple samples. These results suggest that neurosymbolic verification is a viable and precise strategy for deploying LLMs reliably in high-stakes closed-domain applications.

Keywords: hallucination detection, large language models, knowledge graphs, neurosymbolic systems, closed-domain verification

Contents

Acknowledgments	III
Abstract	IV
1 Introduction	1
1.1 Motivation and Research Questions	1
1.2 Research Objective	2
1.3 Proposed Approach: KGVs	2
1.4 Contributions	3
1.5 Thesis Outline	3
2 Background and Related Works	4
2.1 Understanding Hallucinations in LLMs	4
2.1.1 Hallucinations Definition in LLM	4
2.1.2 LLM Hallucinations Types and Categories	5
2.1.3 Causes of LLM Hallucinations from Training Data	5
2.1.4 Hallucination Detection Benchmarks	6
2.2 Knowledge Augmentation Approaches	7
2.2.1 Retrieval-Augmented Generation	7
2.2.1.1 Components	8
2.2.1.2 Models	8
2.2.2 Knowledge Graphs as Structured Knowledge Repositories	8
2.2.2.1 Definition and Representational Framework	8
2.2.2.2 Integration with Large Language Models	9
2.3 Controlling and Constraining LLM Output	9
2.3.1 White-Box Approaches: Leveraging Logit Access	9
2.3.2 Black-Box Control: Neurosymbolic Approaches for Constrained Generation	10
2.4 Black-Box LLM Hallucination Detection	11
2.4.1 Zero-Resource Knowledge Hallucination Detection	11
2.4.2 External Knowledge-Based Hallucination Detection	13
2.5 Research Gaps and Motivation	14
2.6 Chapter Summary	15

3	KGVS: Proposed Framework for Hallucination Detection	16
3.1	System Implementation	16
3.1.1	Hallucination Categories	18
3.1.1.1	Factual Inconsistency Categories	18
3.1.1.2	Logical Inconsistency Categories	19
3.2	Triple Extraction	20
3.3	Triple Predicate Canonicalization	21
3.3.1	Canonicalization Prompt Design	21
3.3.2	Model Selection Experiment	22
3.4	Logical Inconsistency Detection	23
3.4.1	Isolation Test Design	24
3.4.2	Model Evaluation Results	24
3.5	Evidence Retrieval	25
3.6	Factual Inconsistency Detection	26
3.7	Chapter Summary	27
4	Experiment	29
4.1	Experiment Setup	29
4.1.1	Dataset	29
4.1.2	Evaluation Metrics	30
4.1.3	Models	31
4.2	Results	32
4.2.1	LLM Alone vs. LLM with Symbolic Verification	32
4.2.2	System Detection Performance	32
4.2.3	Hallucination Type Analysis	34
4.3	Summary of Research Question Answers	35
4.4	Chapter Summary	36
5	Conclusion and Future Work	37
5.1	Thesis Summary	37
5.2	Limitations	38
5.3	Future work	39
	Appendix	41
A	Knowledge Graph Relationship Types	42
B	Dataset Question Samples	44
	Single-Relationship Questions	44
	Two-Relationship Questions	44
C	System Prompts	46
C.1	Stage 1 — Answer Generation Prompt	46
C.2	Stage 1 — Triple Extraction Prompt	46
C.3	Stage 2 — Canonicalization Prompt	47

C.4	Stage 3 — Logical Inconsistency Detection Prompt	49
C.5	Stage 5 — Pass 1: Symbolic Constraint Check Prompt	50
C.6	Stage 5 — Pass 2: Factual Hallucination Detection Prompt	51
D	Lists	53
D.1	List of Figures	53
D.2	List of Tables	54
	References	55

Chapter 1

Introduction

Language modelling has evolved through four stages, progressing from pure text generation to complex task solving [30]. Models like GPT-4 [27], LLaMA [28], and Gemini [29] can now handle tasks of varying complexity, and results on standard benchmarks demonstrate that these models can be applied to a wide range of fields including medicine, education, mathematical problem solving, and coding. However, the growing capabilities of these models and the increasing reliance on them make it important to identify and address the problems they tend to produce.

1.1 Motivation and Research Questions

LLMs have shown strong performance across different tasks, but they tend to hallucinate by producing nonfactual and logically inconsistent information relative to the source content [1, 2]. This is a critical concern because LLMs produce these hallucinations in a fluent and confident manner, making them difficult to identify without an automatic detection system [2]. Addressing this problem is especially important when LLMs are used in high-stakes domains such as finance, medicine, and legal practice, where hallucinations can lead to severe real-world consequences including financial loss, patient safety risks, and legal misjudgments.

Existing hallucination detection methods fall into two main categories: zero-resource approaches that rely on self-consistency across multiple LLM samples, and external knowledge-based approaches that consult an outside source. Both categories have fundamental limitations that prevent them from reliably detecting all hallucinations in closed domains. A detailed analysis of these limitations, including their consequences for hallucination detection, is given in Chapter 2.

These limitations motivate the central research question of this thesis:

How can symbolic domain knowledge be used to detect logically or factually inconsistent LLM outputs?

Two supporting questions guide the investigation:

1. What kinds of hallucinations can be detected through constraint checking against a structured symbolic knowledge base?
2. Does adding a symbolic reasoning layer significantly reduce undetected errors compared to relying on the LLM alone?

1.2 Research Objective

The objective of this thesis is to design and evaluate a closed-domain hallucination detection system that uses a structured knowledge graph as a symbolic reference to verify LLM outputs through direct triple-to-triple comparison using LLMs. Specifically, this thesis aims to demonstrate that structured symbolic knowledge enables fine-grained categorisation of both factual and logical hallucinations, and that a symbolic verification layer significantly reduces undetected errors compared to relying on the LLM alone.

1.3 Proposed Approach: KGVS

To answer these questions, this thesis proposes KGVS (Knowledge Graph Verification System), a neurosymbolic hallucination detection pipeline that verifies LLM outputs against a structured knowledge graph. Instead of checking against unstructured reference text, relying on self-consistency, or using task-specific NLI models, KGVS performs direct triple-to-triple comparison by using the LLM to perform semantic matching over structured evidence.. Additionally, KGVS explicitly detects logical inconsistencies between triples extracted from the same LLM output and provides fine-grained categorisation of the hallucinations produced, addressing both supporting questions simultaneously.

KGVS operates in a closed domain, and the domain chosen to evaluate the system is consumer credit scoring. This domain was selected because it is a high-stakes setting with clearly formalisable rules that naturally trigger LLM hallucinations, making it well-suited for symbolic verification. KGVS is a five-stage pipeline that begins with triple extraction from the LLM output, followed by predicate canonicalization against the knowledge graph schema, logical inconsistency detection, evidence retrieval from the knowledge graph database, and finally factual inconsistency detection through direct triple-to-triple comparison against the retrieved evidence.

KGVS operates in a closed domain — a bounded subject area whose facts and rules are fully enumerable and can be stored in a structured knowledge base prior to deployment — where the domain chosen to evaluate the system is consumer credit scoring.

1.4 Contributions

The main contributions of this thesis are as follows:

- We propose KGVS (Knowledge Graph Verification System), a neurosymbolic closed-domain hallucination detection system that verifies LLM outputs against a structured knowledge graph through direct triple-to-triple comparison, without relying on NLI models or unstructured text verification.
- We introduce a dedicated logical inconsistency detection stage that identifies three types of logical contradictions between triples extracted from the same LLM output: direct conflict, monotonicity violation, and tier order violation.
- We introduce a fine-grained hallucination categorisation scheme that classifies detected hallucinations into factual inconsistency subtypes (rule violation, fabricated fact and predicate mismatch) and logical inconsistency subtypes (direct conflict, monotonicity violation, and tier order violation).
- We introduce a dedicated factual inconsistency detection stage that identifies factual hallucinations in LLM outputs by comparing extracted triples directly against evidence retrieved from the knowledge graph, without relying on NLI models.
- We evaluate KGVS across four LLMs on a custom domain-specific dataset of 100 questions, demonstrating the effectiveness of structured symbolic knowledge for hallucination detection in closed-domain settings.

1.5 Thesis Outline

The remainder of this thesis is organized as follows:

- Chapter 2 reviews the background on LLM hallucinations, knowledge augmentation approaches, constrained generation methods, and existing hallucination detection approaches covering both zero-resource and external knowledge-based methods.
- Chapter 3 presents the design and implementation of KGVS, detailing each stage of the five-stage pipeline along with the model selection experiments conducted at each stage.
- Chapter 4 presents the evaluation of KGVS across four LLMs on a custom domain-specific dataset, reporting the detection results and analyzing the system performance across different hallucination types.
- Chapter 5 summarizes the findings of this thesis, discusses the limitations of the proposed system, and outlines directions for future work.

Chapter 2

Background and Related Works

This chapter explains the foundations needed to understand the approach used in this thesis: a knowledge-grounded verification layer that detects hallucinations in LLM outputs by checking them against explicit symbolic knowledge. We commence with Section 2.1, where we define hallucinations in LLMs, discuss their types and causes, and review the benchmarks used to evaluate detection approaches. Subsequently, in Section 2.2, we explore Retrieval-Augmented Generation (RAG) as a knowledge augmentation architecture, which motivates our approach where we will replace retrieval with symbolic constraint checking, and we introduce knowledge graphs as the structured knowledge representation that will serve as the backbone of our proposed approach. Following this, Section 2.3 examines how we can control LLM outputs, particularly in black-box settings where control typically occurs after generation, as it has several connections to black-box hallucination detection. We include white-box approaches in this section primarily to compare the different methodologies and understand how they differ. In Section 2.4, we finally discuss papers that present approaches for detecting hallucinations in black-box LLMs, covering both zero-resource and external knowledge-based methods.

2.1 Understanding Hallucinations in LLMs

This section covers the foundational concepts of LLM hallucinations, including their definition, types, causes, and the benchmarks used to evaluate detection approaches.

2.1.1 Hallucinations Definition in LLM

In the current time, while we are in an era of rapid evolution in LLMs that generate fluent responses across various tasks, there is a critical problem that may have a significant impact on certain crucial fields where the information provided in the responses of different LLMs must be factual. Alongside the evolution of LLMs and their fluent responses, LLMs remain prone to hallucinations. LLM hallucination refers to the phenomenon where

an LLM produces responses that are unfaithful or nonsensical relative to the source content [1, 2]. Furthermore, hallucinations have become increasingly difficult to detect, as LLMs produce fluent responses, which may lead to harmful consequences.

2.1.2 LLM Hallucinations Types and Categories

First, there are two traditional types of hallucinations: intrinsic and extrinsic [2]. Intrinsic hallucination means that the generated output contradicts the source content — for example, when summarizing a text and contradicting a date mentioned in the original content. Extrinsic hallucination means that the generated output cannot be verified against the source content or an external knowledge base. Second, in more recent papers, there is a further categorization of LLM hallucination types into factuality hallucinations and faithfulness hallucinations, each of which is further subdivided [1]. For factuality hallucinations, there are two types: the first is factual contradiction, where the LLM generates output that contradicts facts grounded in reality, and the second is factual fabrication, which refers to generated output that cannot be verified against any known source. As for faithfulness hallucinations, they are categorized into three types. The first is instruction inconsistency, where the LLM output deviates from the user’s given directions. The second is context inconsistency, where the LLM output is unfaithful to the contextual information provided by the user. The third is logical inconsistency, where the LLM exhibits internal logical contradictions within its own output.

2.1.3 Causes of LLM Hallucinations from Training Data

In this section, we explore the root cause of hallucinations in LLMs from data [1, 2].

The data used for training LLMs comprises two primary categories: (1) pre-training data, through which LLMs acquire most of their general knowledge and factual information, and (2) alignment data, which teaches LLMs to align with human preferences [1].

Misinformation and Biases Fake news and unfounded data have spread widely across the internet, comprising a significant portion of LLM training data and consequently causing hallucinations. Additionally, certain biases related to hallucination have been observed, particularly those associated with gender and nationality [1].

Knowledge Boundaries Although LLMs are equipped with extensive factual knowledge, they inherently possess knowledge boundaries. The long-tail boundary arises from the non-uniform distribution of training data, which causes LLMs to demonstrate varying levels of proficiency across different domains. The up-to-date problem occurs because once an LLM is trained, its knowledge becomes outdated, and factual information that changes over time may lead to hallucinations [1].

Copyright-Sensitive Knowledge Since LLMs rely on publicly available data, limitations arise in domains that contain inaccessible or restricted data, which causes hallucinations in those specific domains [1].

Inferior Alignment LLMs struggle to acquire new knowledge during Supervised Fine-Tuning (SFT), and a correlation has been identified between the knowledge acquired during SFT and the occurrence of hallucinations [1].

2.1.4 Hallucination Detection Benchmarks

Previous benchmarks were focused on models with specific tasks and were not general purpose. The benchmarks were evaluating smaller and task-specific models which produce errors which are different from new era LLMs, since benchmarks used were on smaller task-specific models which is not compatible with today’s LLMs and cannot be used to evaluate how well detection methods used on today’s powerful LLMs; these benchmarks must be changed [1].

A set of benchmarks has since been introduced to evaluate how well hallucination detection methods work, in this part we discuss some of the modern datasets to evaluate hallucination detection approaches.

HaluEval: A large-scale benchmark introduced by [16] which is a large collection of 35,000 generated and human-annotated samples to evaluate LLMs’ ability to recognize hallucinations. The benchmark covers 30,000 samples across multiple tasks including question answering, knowledge-grounded dialogue, and text summarization, providing a comprehensive evaluation framework for hallucination detection across different LLM applications, along with 5,000 samples of general user queries with ChatGPT responses.

FELM: A benchmark introduced by [19] for evaluating factuality evaluation methods themselves, rather than directly evaluating LLMs. It contains 2,000+ responses from various LLMs across different domains, each annotated with fine-grained factuality labels, enabling systematic assessment of how well different evaluation metrics and detectors perform at identifying factual errors.

SelfCheckGPT-Wikibio: The WikiBio GPT-3 dataset introduced by [4] offers a sentence-level dataset created by GPT-3 on individuals from the WikiBio dataset, with manual annotation of factuality. WikiBio is a dataset where each input is the first paragraph of Wikipedia articles of specific concepts. They sampled 238 of the top 20 percent of longest articles.

BAMBOO: A benchmark proposed by [17] which is a multi-task long-context benchmark used to evaluate long-text tasks including hallucination detection. BAMBOO consists of 10 different datasets from 5 different long-text understanding tasks: question answering, hallucination detection, text sorting, language modeling, and code completion.

ScreenEval: A benchmark introduced by [18] for evaluating factual inconsistencies in long dialogues. This dataset comprises 52 dialogues averaging 6,000 tokens per dialogue. The dataset uses TV scripts and summaries pulled from the SummScreen dataset; they also generate summaries using Longformer and GPT-4 on 52 scripts from the SummScreen test set.

PHD: A passage-level hallucination detection benchmark introduced by [20] containing 300 samples generated by ChatGPT on Wikipedia articles, with human annotations at both sentence and passage levels. The paper also introduces a reverse validation method that reconstructs questions from generated passages and verifies consistency, specifically designed to address the omission problem in traditional consistency-based detection.

LSum: A benchmark introduced by [21] for evaluating hallucination in text summarization tasks. The paper proposes an adversarial approach that decouples comprehension and embellishment abilities of LLMs, demonstrating that factual consistency can be improved by separating what the model understands from what it generates, with applications for both detection and mitigation of hallucinations in summarization.

2.2 Knowledge Augmentation Approaches

Large language models store factual knowledge implicitly in their parameters, which makes it difficult to update, verify, or constrain that knowledge after training. Knowledge augmentation addresses this limitation by providing explicit, external knowledge during generation or verification. This section reviews two complementary paradigms: retrieval-based augmentation using unstructured text (Retrieval-Augmented Generation) and structured knowledge repositories in the form of knowledge graphs. Both approaches inform the design of KGVs, which replaces traditional retrieval with symbolic constraint checking against a knowledge graph.

2.2.1 Retrieval-Augmented Generation

Pre-trained language models have been shown to learn a substantial amount of in-depth knowledge from data, and they are capable of doing so without accessing any external memory. However, such models cannot expand or revise their knowledge after training, which may lead to hallucinations [3]. This thesis draws on the same motivation as RAG — grounding LLM outputs in external knowledge — but replaces retrieval with symbolic constraint checking against a structured knowledge representation.

2.2.1.1 Components

RAG consists of two core components: the Retriever and the Generator [3].

- **Retriever:** Responsible for returning the top- k documents based on a truncated distribution over the input sequence X .
- **Generator:** Utilizes the retrieved documents as additional context to generate the target sequence Y .

2.2.1.2 Models

- **RAG Sequence Model:** The RAG sequence model uses the same set of retrieved documents to generate the entire target sequence [3].
- **RAG Token Model:** The RAG token model uses different retrieved documents for each individual target token, allowing for more fine-grained document selection during generation [3].

2.2.2 Knowledge Graphs as Structured Knowledge Repositories

While RAG retrieves unstructured text passages, knowledge graphs offer a structured alternative where facts are explicitly represented as triples. This structure enables deterministic querying, logical inference, and direct triple-to-triple comparison — features that are particularly valuable for hallucination detection. The following subsections define the representational framework of knowledge graphs and review how they have been integrated with large language models.

2.2.2.1 Definition and Representational Framework

Knowledge graphs represent information as entities (nodes) and relations (edges), forming a structured repository of facts. Facts are stored as triples of the form $\langle \textit{subject}, \textit{predicate}, \textit{object} \rangle$, such as $\langle \textit{Paris}, \textit{capitalOf}, \textit{France} \rangle$, which allows structured reasoning over the stored knowledge.

To illustrate, consider the sentence "Paris is the capital of France." This would be stored as the triple (Paris, capitalOf, France). If an LLM generates the response "Paris is the capital of Germany," a verification system can query the knowledge graph for (Paris, capitalOf, ?). The graph returns "France," which contradicts the generated "Germany," signaling a hallucination. This query-based mechanism is how knowledge graphs enable fact-level verification.

In the context of hallucination detection, knowledge graphs are used as a way of storing rules or facts that will be verified or assigned a score [6, 8, 12].

2.2.2.2 Integration with Large Language Models

Recent works integrate knowledge graphs with LLMs to improve factual accuracy and detect hallucinations. The integration typically follows two patterns. The first pattern constructs graphs from LLM responses and uses them for structural consistency checking. For instance, [12] constructs graphs from responses and uses graph neural networks to model contextual dependencies between triples, enabling detection of logical inconsistencies across multiple facts. The second pattern uses knowledge graphs as an external reference to verify extracted facts. [6] and [8] leverage knowledge graphs for fine-grained fact-level consistency checking by comparing triples extracted from LLM outputs against triples stored in the graph.

2.3 Controlling and Constraining LLM Output

Enforcing domain rules and logical constraints on LLM output is a major concern in modern research, as it makes LLMs more reliable. While this thesis focuses on detecting hallucinations after generation, examining methods that prevent hallucinations through controlled generation provides valuable context. Both control and detection share a common goal: ensuring LLM outputs align with expected facts or rules. Additionally, some detection approaches borrow techniques from constrained decoding, particularly in how they interact with external knowledge sources. There are several approaches for controlling LLM output: ones that are used with white-box LLMs and ones that are used with black-box LLMs, where access to their logits is unavailable. In this section we discuss each one of them, with emphasis on black-box methods as they are most relevant to the detection approaches surveyed in Section 2.4.

2.3.1 White-Box Approaches: Leveraging Logit Access

The techniques used to control LLM output share the same goal: enforcing $p(x|a)$, where x is the output sequence and a is the constraint that needs to be satisfied.

There are papers that distill a TPM (Tractable Probabilistic Model), most commonly an HMM (Hidden Markov Model), then combine the HMM with the LLM’s output probability distribution and search ahead to determine whether the given constraint will be enforced. One such paper, GeLaTo, trains an HMM via maximum likelihood estimation on samples drawn from LLM output, and at generation time combines the HMM probability with the LLM output [10]. Another paper, Ctrl-G, generalizes GeLaTo by distilling an HMM, constructing a DFA, and then conditioning the HMM on the DFA [5]. Yet another paper distills an HMM and combines it with an efficiently trained classifier that guides the computation of the Expected Attribute Probability, which in turn steers generation [9].

Another approach is presented in the paper FUDGE, which shares the same goal of modeling $p(x|a)$, but instead of using a TPM, it trains a binary predictor on whether

constraint a will be satisfied based on an incomplete sequence prefix [7]. It multiplies the probability distributions of the predictor and the LLM output, then selects the token with the highest weight.

These white-box methods share a common pattern: they all require access to the LLM’s internal logits and involve training auxiliary models (HMMs or binary predictors) to guide generation. While they are effective, their reliance on logit access limits their work with black-box LLMs.

2.3.2 Black-Box Control: Neurosymbolic Approaches for Constrained Generation

In the previous section, constraints were applied by leveraging access to the LLM’s probability distribution. However, when using black-box LLMs such as ChatGPT or Claude, access to the model’s internal logits is unavailable, so alternative approaches are needed to enforce rules or logical constraints on the output of such black-box LLMs.

One such technique is presented in the paper SketchGCD, which proposes an approach for constrained decoding on LLM output that requires no further training [13]. It uses a lightweight open-source model to refine the output of a heavyweight black-box LLM to satisfy specific constraints. This approach is divided into two components: a sketcher and a constrained decoder. For the constraints to be applied, it restricts entities and relations to the Wikidata knowledge graph and enforces a structural format that the output must follow.

Another technique that integrates LLMs with symbolic solvers to improve logical problem-solving is LOGIC-LM [22], which takes a problem P and a goal G and feeds them into a multi-stage process to derive the answer. First, it prompts an LLM to translate P and G into task-specific symbolic representations, using four different symbolic formulations: logic programming, first-order logic, constraint satisfaction problems, and analytical reasoning. After converting P and G into symbolic representations, they are passed to a symbolic solver to obtain answer A by enforcing logical rules. The approach also includes a self-refinement module: if the solver returns a logical or syntactic error, the LLM revises the formulation and repeats the process iteratively.

A third approach used to control the generated content at the token level of black-box LLMs introduces ScoPE (Score-based Progressive Editor), proposed by [24]. This method employs an editor module that edits blocks of size b tokens from the text generated by a black-box LLM, evaluating how well a target attribute is incorporated into the text. ScoPE follows an iterative process of progressively editing the output text to align it with the target domain, using score functions based on fine-tuned masked language models and task-specific discriminators. The editor operates without access to model parameters or logits, making it compatible with commercial LLM APIs, and is trained to maximize the disparity in target domain scores between input and edited text while preserving fluency through techniques such as teacher forcing and selective token-position training [24].

A further example worth noting is the correction component of GraphEval [23] GraphCorrect, which demonstrates that post-hoc repair can follow directly from detection output; both the detection and correction contributions of this paper are discussed together in Section 2.4.

These black-box control methods demonstrate that logical and factual constraints can be enforced on LLM outputs without internal logit access. Notably, approaches like Logic-LM integrate symbolic solvers for logical verification, while SketchGCD relies on knowledge graphs (Wikidata) as the source of constraints—the same type of structured knowledge that this thesis uses for detection. Additionally, ScoPE shows that score-based editing with domain-specific MLMs offers another viable path for enforcing constraints, such as topic or sentiment control, without external knowledge resources. GraphCorrect adds a further dimension by demonstrating a post-hoc correction mechanism that first detects hallucinations and then repairs them using structured context, effectively bridging the gap between detection and correction. The key difference across these methods is one of timing and mechanism: control methods intervene during generation to prevent hallucinations (whether through sketching, symbolic solving, or progressive editing), while methods like GraphCorrect operate after generation to identify and fix them. This distinction aside, the underlying goal of ensuring LLM outputs align with expected facts or rules is common to all, making constrained decoding a relevant adjacent field to the hallucination detection approaches discussed next.

2.4 Black-Box LLM Hallucination Detection

As mentioned previously, hallucinations are a major concern in the LLM field, so detecting them is a crucial area of research in which several approaches have been proposed, each employing different techniques. The hallucination detection approaches for black-box LLMs are divided into two parts: zero-resource and external knowledge hallucination detection.

2.4.1 Zero-Resource Knowledge Hallucination Detection

This section covers four different types of zero-resource approaches: 1) SelfCheck zero-resource hallucination detection, 2) fine-grained cross-model evaluation, 3) graph-based context-aware hallucination detection and 4) knowledge graphs for hallucination self-detection. These methods share a common strength: they require no external knowledge base, making them broadly applicable across domains. However, they also share a fundamental limitation that motivates the external knowledge-based approaches discussed in Section 2.4.2.

SelfCheckGPT: One approach is presented in the paper SelfCheckGPT, which requires no external knowledge and uses the LLM to self-check its output, operating effectively with black-box LLMs [4]. This is achieved by obtaining a response R from

the LLM given a prompt, then sampling additional responses, which are subsequently used with different methods (BERTScore, NLI, n-grams, etc.) to measure the consistency between the original response and the samples, and then assigning a hallucination score.

FINCH-ZK: Another approach is presented in the paper FINCH-ZK (FINE-grained Cross-model consistency for Hallucination detection and mitigation with Zero Knowledge) [11]. It requires no external knowledge base to detect hallucinations and uses an LLM to judge the outputs. First, a cross-model sample generation step is performed, in which variants of the prompt are generated and a set of sampler LLMs (M) are used to produce samples. Once these samples are collected, a judge LLM performs fine-grained evaluation. The response is segmented into blocks, and then a judge LLM (J) is used to evaluate each block against each sample, after which hallucination scores are computed.

Graph-based Context-Aware (GCA): The third approach is Graph-based Context-Aware (GCA) zero-resource hallucination detection, which focuses on long text generation with black-box models [12]. The paper begins by extracting knowledge triples from the response by providing the LLM with the response and prompting it to extract the triples. Next, it asks the LLM to verify each triple against the original response, and if there is ambiguity in equivalence, it asks the LLM to adjust. Then they construct a graph for the original response and each sampled response, using a Relational Graph Convolutional Network (RGCN) which passes information through nodes, so that when the model checks a triple, it remembers information from other triples and can detect logical errors. After that, they perform graph-based consistency comparison where they compare triples from the original response with triples from sampled responses. If alignment is found, they add 1 to the consistency score, and if there is contradiction between triples, they subtract 1 from the consistency score. Finally, they perform reverse verification through triple reconstruction, where they check if the LLM can correctly reconstruct the head entity, relation, and tail entity of each triple using three different tasks, and combine these scores with the consistency score to make the final hallucination judgment.

Knowledge Graph Self-Detection: The fourth approach involves using knowledge graphs for self detection of hallucinations. One method is presented in the paper Fact-SelfCheck, which operates at the fact level rather than the sentence level [6]. It follows a similar procedure to SelfCheckGPT by first generating a response and multiple samples. However, instead of comparing sentences, it constructs knowledge graphs by extracting entities and relations from the response using LLM prompts, does the same with the samples, and then checks the consistency of the extracted facts to assign hallucination scores. Similarly, another approach is presented in the paper Lie to Me, which enhances both single-sample and multi-sample self-detection techniques using knowledge graphs [8]. After obtaining a response, it prompts the LLM to construct a knowledge graph, and then uses either self-questioning or self-confidence approaches for single-sample detection, or integrates knowledge graphs into SelfCheckGPT for multi-sample detection, to assign hallucination scores at the fact level.

While these zero-resource approaches offer practical advantages, they share a critical limitation: they cannot verify claims that fall outside the LLM’s parametric knowledge.

If a model confidently generates a false fact and also repeats it in sampled responses, consistency-based methods may incorrectly label it as factual. This limitation necessitates approaches that consult external knowledge sources, which are discussed next.

2.4.2 External Knowledge-Based Hallucination Detection

While zero-resource approaches offer practical advantages, they cannot verify claims that fall outside their parametric knowledge, which is where external knowledge-based methods are needed.

FactScore: One approach is proposed in the paper FActScore, which evaluates generation based on atomic facts [14]. This approach uses both human annotation and automated annotation to evaluate atomic facts. For the human annotation, they sampled 183 entities from Wikipedia, fed prompts to LLMs, and then human annotators broke the generation into atomic facts and labeled them as supported, not-supported, or irrelevant based on the knowledge source of Wikipedia. However, this approach is expensive, so they introduced automated annotation where an LLM subject generates text and an LLM evaluator assigns fact scores. The estimator breaks the generation into atomic facts and assigns scores against the knowledge source.

REFCHECKER: Another approach is REFCHECKER, which uses claim-triplets instead of sentence-level evaluation, as one sentence may contain several facts [15]. This approach consists of two parts: the extractor and the checker. The extractor extracts triples from the response using models like GPT-4, Mixtral, or a fine-tuned Mistral model trained on 10k responses with Mixtral as a teacher. The checker then compares these triples against a reference to determine if they are entailed, contradicted, or neutral.

GraphEval and GraphCorrect: There is an approach that uses a knowledge graph and the external knowledge from a given context to match triples [23]. GraphEval focuses on closed-domain hallucination detection, where it first constructs a knowledge graph from the given LLM output, then iterates through the triples and identifies whether they are factually consistent with the given context or not. It uses an NLI model in the second stage of checking triples against the context. The same paper [23] also introduces GraphCorrect, a correction component that builds on GraphEval’s detection output. GraphCorrect proceeds in two stages. In the first stage, hallucinated triples are provided to the LLM along with the structured context, prompting it to correct the hallucination and output a newly generated triple. In the second stage, the original triple and its corrected counterpart are taken together with the initial LLM output, and the information from the original triple is replaced with the corrected version in the initial LLM output.

FacTool: A fourth method for detecting factual errors in LLM outputs is introduced in this paper [25]. It works through five stages, starting with claim extraction via an LLM. From there, FacTool generates queries for each claim to pull in evidence from external tools — search engines, code interpreters, calculators, or academic search — depending on the task at hand. Each claim is then given a binary factuality label based on how

relevant the gathered evidence is, with the LLM also offering reasoning, flagging errors, and suggesting corrections for any hallucinated claims.

FacTool brings an extra layer to external knowledge-based hallucination detection by combining web search with task-specific tools, though it still depends on verifying against unstructured text. The approaches discussed above all point to the same takeaway: consulting external knowledge is genuinely useful for detecting hallucinations. That said, two of these methods share a notable limitation — they check claims against unstructured text (Wikipedia), which requires natural language understanding to make the comparison work. GraphEval, despite its use of knowledge graphs, runs into a similar issue: it relies on an NLI model to check triples against unstructured contextual text rather than a structured symbolic knowledge base, meaning the ambiguity of natural language interpretation is still present. This thesis takes a different path by using a structured knowledge graph as the reference point, which makes direct triple-to-triple matching possible and sidesteps the interpretive uncertainty that comes with natural language. This distinction is at the heart of the proposed approach laid out in the next chapter.

2.5 Research Gaps and Motivation

Taken together, the surveyed approaches expose three limitations that motivate the design of KGVS, and that KGVS directly addresses:

- L1 — Dependence on LLM sampling.** Zero-resource approaches such as SelfCheckGPT [4], FINCH-ZK [11], FactSelfCheck [6] and Graph-based Context-Aware (GCA) [12] detect hallucinations by checking consistency across multiple LLM samples. If the model hallucinates the same fact repeatedly, the system fails to flag it. KGVS eliminates this dependency entirely: triples are verified directly against domain rules stored in a structured knowledge graph, not against other LLM-generated samples.
- L2 — Verification against unstructured text.** Approaches that use external knowledge compare LLM outputs against unstructured reference text, requiring natural language understanding to bridge the semantic gap between the generated output and the reference [14, 25]. Approaches that use structured knowledge do not fully escape this problem: GraphEval [23] constructs a knowledge graph but still compares extracted triples against unstructured contextual text, meaning the ambiguity of natural language interpretation is reintroduced even when the knowledge source is structured. The design that covers this is detailed in Chapter 3, specifically in the evidence retrieval and factual inconsistency detection stages.
- L3 — Absence of logical inconsistency detection and fine-grained categorisation.** No surveyed approach explicitly targets logical inconsistency as a separate category, such as contradictions between triples generated in the same response, monotonicity

violations, or tier ordering violations. Furthermore, no surveyed approach categorises detected hallucinations into diagnostically distinct types. KGVS addresses both shortcomings simultaneously: it introduces a dedicated logical inconsistency detection stage covering Direct Conflict, Monotonicity Violation, and Tier Order Violation, and it provides a structured taxonomy of factual inconsistency subtypes — Rule Violation, Predicate Mismatch, and Fabricated Fact — enabling fine-grained diagnostic feedback on the nature of each detected error. This design is formally defined and evaluated in Chapter 3 and Chapter 4 respectively.

These three limitations collectively motivate the central design decisions of KGVS, which is presented in Chapter 3.

2.6 Chapter Summary

The literature reviewed in this chapter collectively motivates the design of KGVS. Hallucination detection approaches are broadly split into zero-resource methods, which are fundamentally limited by their reliance on LLM sampling consistency, and external knowledge-based methods, which either verify against unstructured text or retain NLI dependence even when structured knowledge is involved. Neither category explicitly targets logical inconsistencies within a single LLM response, nor provides fine-grained categorisation of detected errors. These three gaps directly shape the design decisions presented in the following chapter.

Chapter 3

KGVS: Proposed Framework for Hallucination Detection

This chapter presents the design and implementation of KGVS (Knowledge Graph Verification System), a neurosymbolic hallucination detection system for closed-domain settings. KGVS combines the reasoning capabilities of LLMs with structured domain knowledge stored in a knowledge graph to detect hallucinations produced by LLMs. The system focuses specifically on identifying factually and logically inconsistent outputs.

KGVS is designed to directly address the three limitations identified in Section 2.5: dependence on LLM sampling (L1), verification against unstructured text and residual NLI dependence (L2), and absence of logical inconsistency detection and fine-grained categorisation (L3). The following sections describe how each design decision maps to these limitations.

Following the taxonomy of [33], KGVS qualifies as a Type 3 neurosymbolic architecture — termed *Neuro; Symbolic* (Cooperative) — in which a neural component and a symbolic component each specialize in complementary tasks and interact via structured data exchange. The neural components are the LLMs responsible for triple extraction, predicate canonicalization, and verdict assignment in the verification passes. The symbolic components are the Neo4j knowledge graph, the deterministic Cypher queries used during evidence retrieval, and the explicit domain constraint predicates applied during factual verification.

3.1 System Implementation

The domain rules and constraints are stored in a graph database (Neo4j), which consists of nodes and relationships and serves as the knowledge base against which LLM outputs are verified. The domain captured in this graph is consumer credit scoring, covering rules such as credit score to interest rate mappings, risk tier assignments, approval odds,

and related eligibility constraints. This domain was selected because its rules are clearly formalisable and unambiguous, making it well-suited for symbolic verification.

The system is implemented using LangChain, which provides a straightforward pipeline abstraction sufficient for this task. LangGraph was considered as an alternative; however, since the pipeline does not require cycles or conditional branching between stages, the additional complexity it introduces was unnecessary. A structured state dictionary carries the information produced at each stage forward through the pipeline.

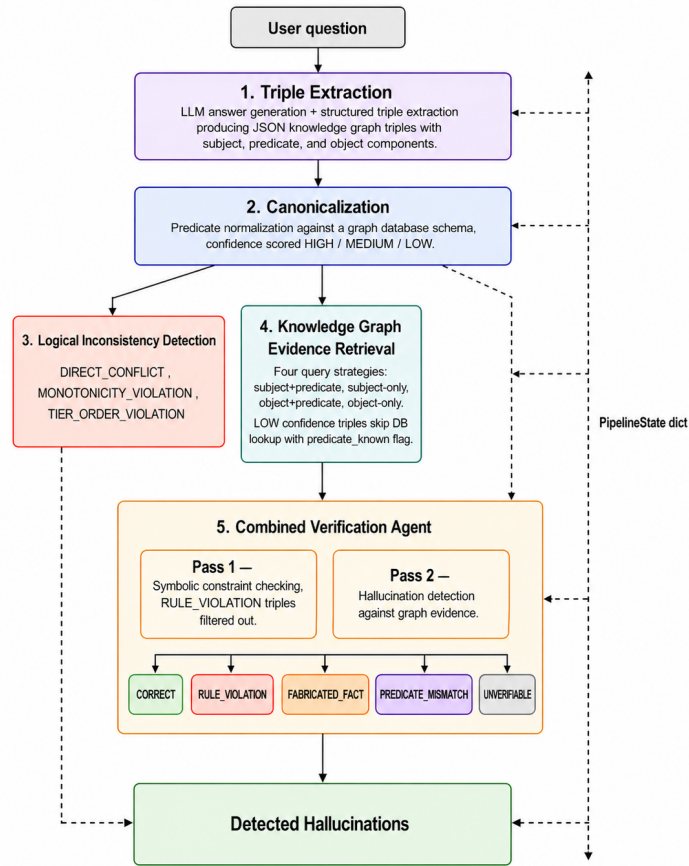


Figure 3.1: Overview of the hallucination detection pipeline. The system processes the user query through five stages: triple extraction, canonicalization, logical inconsistency detection, evidence retrieval, and combined verification (symbolic constraint check followed by factual detection).

Given a user query containing domain context and a question, the query is processed through five main stages to detect hallucinations. First, the user input is passed to an LLM to produce a structured answer; a second, dedicated LLM then extracts triples from that answer, ensuring that any information, whether structured or unstructured, is captured as the necessary triples. Second, the extracted triples are passed to the

canonicalization stage, where an LLM matches each triple’s predicate to the closest predicate in the graph database schema, either canonicalizing it via semantic similarity or marking it as unmatched. Third, the canonicalized triples are examined for internal logical inconsistencies. Fourth, the canonicalized triples are used to retrieve related triples from the database and to fetch the domain constraints, which represent the general rules of the domain. Fifth, the detection of factually inconsistent triples proceeds in two passes: the first pass compares the triples against domain constraints, and the second pass verifies them against the related triples retrieved earlier. The system architecture is depicted in Figure 3.1.

3.1.1 Hallucination Categories

Based on the symbolic verification approach, each extracted and canonicalised triple (s, p, o) is assigned to one of two high-level inconsistency types: *factually inconsistent* or *logically inconsistent*. The classification follows a decision flow that first checks for logical inconsistency between multiple triples and then separately verifies factual correctness against the domain constraints and the graph database.

Triples whose predicate is completely absent from or unknown to the established knowledge graph schema during canonicalization are classified as **Unverifiable**. Because the schema lacks a matching relationship type for these predicates, the system lacks a symbolic baseline to either confirm or refute the underlying claim.

3.1.1.1 Factual Inconsistency Categories

Each factually inconsistent triple is assigned to one of three categories depending on the nature of the inconsistency detected. These categories are mutually exclusive and ordered by severity: a triple is assigned the first category whose condition it satisfies.

A. Rule Violation The triple’s object violates one of the domain rules stored as constraints in the knowledge graph. These rules define the permissible value ranges and type restrictions for each predicate in the domain — for example, an interest rate of 890% falls outside the permissible domain range and is therefore a Rule Violation regardless of whether the subject entity exists in the database. This is the most severe category, as the generated value is not merely unsupported but actively contradicts a formal domain constraint.

B. Predicate Mismatch The subject s and predicate p are both present in the database schema, and the object o is a legal value according to domain constraints, but no triple (s, p, o) exists in the graph database. That is, the predicate is a known relation type but is not associated with this particular subject in the knowledge base. The triple is structurally plausible but factually incorrect for this subject.

C. Fabricated Fact The triple asserts a relationship involving one or more entities that do not exist in the domain knowledge base, while the predicate itself is a known relation type and the object is a legal value. Three subcases are distinguished based on which component is fabricated:

- The subject s is not a known entity in the database, while the predicate p and object o are both supported.
- The object o is not a known entity in the database, while the subject s and predicate p are both supported.
- Neither the subject s nor the object o is a known entity in the database, while the predicate p is supported.

3.1.1.2 Logical Inconsistency Categories

Logical inconsistencies are identified without querying the graph database; they arise from contradictory relations among the triples generated from a single LLM answer. These checks are performed after canonicalization but before factual verification. The following three categories are defined.

A. Direct Conflict Two triples share the same subject s and the same predicate p but assign two different, mutually exclusive objects o_1 and o_2 . *Example:* (600, HAS_INTEREST_RATE, 8%) and (600, HAS_INTEREST_RATE, 15%).

B. Monotonicity Violation A predicate that must respect a monotonic relationship with credit score is violated across different subjects. For instance, interest rate must decrease as credit score increases, so a higher-scored borrower carrying a higher rate than a lower-scored one constitutes a violation. *Example:* A credit score of 700 is assigned an interest rate of 9.5%, while a score of 580 is assigned 7.2%.

C. Tier Order Violation A lower credit score receives a better (lower-risk) tier than a higher credit score, contradicting the fundamental ordering of creditworthiness tiers (*deep subprime* < *subprime* < *near prime* < *prime* < *super prime*). *Example:* Score 500 is labelled “super prime” (best tier), whereas score 750 is labelled “subprime” (worst tier).

Triples that pass both factual and logical consistency checks are considered *consistent* with respect to the symbolic domain knowledge. All other triples are flagged as hallucinations of the corresponding category, providing fine-grained feedback on the nature of the LLM’s error.

3.2 Triple Extraction

In this stage, a primary LLM (Mistral) first takes the user query and produces a structured answer. Mistral was chosen as the fixed answer generator to ensure a consistent and reproducible baseline across all extraction model comparisons; fixing the answer source guarantees that any differences observed in Table 3.1 reflect the extraction model’s capability rather than variation in the generated answer itself. However, because the answering model may embed extra explanations or unstructured text alongside the requested structured output, a second, dedicated extraction LLM is employed. Its task is to guarantee that the final output consists solely of the necessary triples. The stage is deliberately split into answering and extraction because a single LLM asked to handle both tasks simultaneously tends to hallucinate and produce incorrect outputs. The pipeline therefore proceeds as follows: the first LLM generates an answer, and the second LLM extracts all pertinent triples from that answer.

The extraction follows the Extract-Define-Canonicalize (EDC) framework proposed by [26]: the LLM first identifies unique entities mentioned in the answer, then finds the predicates that link those entities, and finally produces a natural-language definition for each predicate. To select the most reliable extraction model, five candidates were evaluated — GLM-4.7, LLaMA 3.3:70B, GPT-4.1, Gemini-2.5-Flash, and Mistral — while keeping Mistral as the fixed answer generator. Five representative questions were used, each targeting a specific part of the domain context — such as interest rate mappings, risk tier assignments, and approval odds — so that together they cover most of the domain context. The expected triples for each question were defined manually before running any model, ensuring that the evaluation was not influenced by model outputs. Table 3.1 summarises the results. Each cell indicates how many of the expected triples were correctly extracted, and whether any additional correct triples or incorrect extractions were produced.

After comparing the outputs, GLM-4.7 was selected as the extraction model for the final pipeline for several reasons. First, it consistently returned the exact set of required triples and never fabricated an incorrect triple across all five questions. Second, on Q2 it contributed extra correct triples beyond the expected set: some were extracted from the LLM’s explanatory description that accompanied the structured answer, and others arose from triples whose object field contained multiple values that GLM-4.7 correctly decomposed into separate triples. While such additional extractions could in principle introduce noise, this risk is controlled downstream by the canonicalization stage, which filters out any triple whose predicate cannot be matched to the graph schema. This behaviour is therefore beneficial for thorough hallucination detection rather than problematic. Third, although GPT-4.1 also produced zero incorrect triples, GLM-4.7 was preferred because it is open-source, significantly cheaper to operate at scale, and demonstrated the additional extraction capability on Q2 that GPT-4.1 did not. These combined factors make GLM-4.7 the most suitable choice for the extraction stage.

Table 3.1: Extraction model comparison on five pilot questions. “Correct” indicates the number of required triples that were extracted exactly. Additional notation: *add.* = additional correct triples extracted beyond the expected set; *err* = incorrectly extracted triples (hallucinated or malformed predicates); *corr (expl.)* = correct triples extracted from the LLM’s explanatory text accompanying the structured answer, rather than from the structured answer body itself; *dup* = duplicate triple present in the extracted set.

Question	Expected triples	GPT-4.1	Mistral	LLaMA 3.3	GLM-4.7	Gemini-2.5-Flash
Q1	9	9 correct	9 correct	9 correct	9 correct	9 correct
Q2	7	7 correct	7 correct +1 add. +7 err	7 correct	7 correct +1 add. +2 corr (expl.)	7 correct +1 add. +6 err
Q3	15	15 correct	15 correct +4 err	15 correct	15 correct	15 correct
Q4	21 (incl. 1 dup)	21 correct (with dup)	20 correct (no dup)	21 correct (with dup)	20 correct (no dup)	20 correct (no dup)
Q5	18	18 correct	18 correct	18 correct	18 correct	18 correct

3.3 Triple Predicate Canonicalization

Continuing the EDC pipeline [26], this stage takes the extracted triples and matches their predicates to the formal relationship types defined in the database schema. All valid relationship types are fetched from the database at runtime. For each triple, its predicate is submitted to an LLM, which identifies the most semantically aligned relationship type from the schema. Since not every extracted predicate has a corresponding schema entry, the model also returns a match confidence level — *High*, *Medium*, or *Low* — where *Low* explicitly signals that no honest match could be found and the triple should be flagged for downstream hallucination detection rather than discarded.

3.3.1 Canonicalization Prompt Design

The canonicalization prompt instructs the model to follow a structured three-step matching procedure. First, qualifying adjectives that narrow meaning without a dedicated schema counterpart — such as certainty markers (*estimated*, *confirmed*), temporality markers (*current*, *previous*), and ordinality markers (*minimum*, *maximum*) — are stripped from the extracted predicate to expose its core verb phrase. Second, the core predicate is matched against the schema using exact match, synonym inference, or partial overlap, in that priority order. Third, if a qualifier was stripped to achieve a match, its semantic content is preserved by appending it as a parenthetical annotation to the object field, ensuring no information is silently lost. A strict rejection rule prevents forcing a match: if the core verb phrase shares no overlap with any schema relationship type after all steps, the predicate is returned unchanged in ALL_CAPS_WITH_UNDERSCORES form with a *Low* confidence assignment.

3.3.2 Model Selection Experiment

To identify the most reliable canonicalization model, five candidates were evaluated — GPT-4.1, Mistral, LLaMA 3.3 70B, GLM-4.7, and Gemini-2.5-Flash — while keeping Mistral as the fixed answer generator and GLM-4.7 as the fixed extraction model. The evaluation used a controlled set of 30 synthetic triples, stratified across four difficulty categories designed to probe distinct failure modes:

- **Category A — Direct synonyms** (10 triples): predicates that paraphrase a schema relationship closely enough that an exact or near-exact match is expected.
- **Category B — Partial or ambiguous** (5 triples): predicates carrying qualifying adjectives or indirect phrasing that require qualifier stripping or synonym inference to match.
- **Category C — Clear fabrications** (8 triples): predicates describing concepts entirely absent from the schema, such as historical provenance or demographic associations, which should receive *Low* confidence.
- **Category D — Near-schema traps** (7 triples): predicates that superficially resemble schema relationships — sharing surface-level financial vocabulary — but encode distinct concepts not represented in the database, designed to elicit false-positive matches.

Categories A and B form the 15 matchable triples, while Categories C and D form the 15 that should receive *Low* confidence. Each model was evaluated on two metrics: *match decision accuracy*, measuring whether the model correctly matched or rejected each predicate, and *predicate accuracy*, measuring whether matched predicates were mapped to the correct schema relationship type.

Table 3.2: Canonicalization model evaluation results across 30 test triples.

Model	Match Decision Accuracy	Predicate Accuracy
GPT-4.1	30/30 (100%)	15/15 (100%)
Mistral	30/30 (100%)	15/15 (100%)
LLaMA 3.3 70B	30/30 (100%)	15/15 (100%)
GLM-4.7	30/30 (100%)	15/15 (100%)
Gemini-2.5-Flash	30/30 (100%)	15/15 (100%)

As shown in Table 3.2, all five models achieved perfect scores on both metrics. Two factors likely explain this outcome. First, the relationship types stored in the database are self-describing: rather than short identifiers such as `HAS_RATE` or `HAS_TIER`, each relationship type encodes its full semantic meaning directly in its name — for example, `INTEREST_RATE_CHARGED_AS_ANNUAL_BORROWING_COST_PERCENTAGE`. This makes the

matching task substantially easier than what real-world scenarios would be, because even near-synonym or partially overlapping predicates can be resolved by reading the full relationship name, reducing the ambiguity that would arise with abbreviated schema identifiers. Second, the extraction stage produces a natural-language predicate definition alongside each triple, which provides the canonicalization model with additional semantic context beyond the raw predicate string itself. Together, these two factors likely made the task well-constrained enough that model architecture had no measurable effect on accuracy within this candidate set.

Since all models performed identically, the canonicalization model was selected on the basis of cost and latency rather than accuracy. This finding confirms that smaller and cheaper models are entirely sufficient for this stage, which is itself a useful practical result for deployment. Based on these criteria, Mistral was selected as the canonicalization model, as it offers the best cost-per-call ratio among the evaluated candidates while being the same model used for answer generation, reducing the number of API dependencies in the deployment pipeline.

3.4 Logical Inconsistency Detection

In this stage, canonicalized triples are organised into structured groups to identify logical inconsistencies between them. Three types of logical inconsistency are detected. First, **Direct Conflict** occurs when two triples share the same subject and predicate but carry semantically incompatible objects — for example, a credit score of 600 being simultaneously classified as both *prime* and *deep subprime*. Second, **Monotonicity Violation** occurs when a predicate that must respect a monotonic relationship with credit score is violated across different subjects; for instance, since interest rate must decrease as credit score increases, a higher-scored borrower carrying a higher rate than a lower-scored one constitutes a violation. Third, **Tier Order Violation** occurs when the risk tier assigned to a lower credit score is strictly higher in the tier hierarchy than the one assigned to a higher credit score, contradicting the fundamental ordering of creditworthiness tiers (*deep subprime* < *subprime* < *near prime* < *prime* < *super prime*).

The detection pipeline groups the canonicalized triples into three structured sets before passing them to the LLM. The first group, `same.subject.same.predicate`, collects triples that share both the same subject and the same predicate but carry different object values, making them candidates for Direct Conflict detection. The second group, `same.predicate.different.scores`, gathers triples that share a predicate across different credit score subjects and annotates each predicate with its required monotonicity direction (DECREASING, INCREASING, or TIER), providing the LLM with all information needed to reason about monotonicity and tier ordering without consulting the graph database for additional facts. The third group, `all.triples`, provides the complete flat list for reference. The LLM receives all three groups as a single structured JSON payload and is instructed to return only genuine contradictions that exist within the triples themselves, with no appeal to external domain facts.

3.4.1 Isolation Test Design

To evaluate the detection capability of each candidate model in isolation, a controlled test set of 21 canonicalized triples was constructed. The test set was designed to contain exactly 2 Direct Conflicts, 3 Monotonicity Violations, and 2 Tier Order Violations with total of 14 triples forming logical inconsistencies, alongside a control group of 7 consistent triples consist of triples of equal values and triples that that are consistent that should produce no false positives. The injected inconsistencies are described below.

The two **Direct Conflicts** are: (1) credit score 600 simultaneously classified as both *prime* and *deep subprime* under the risk tier predicate, and (2) credit score 700 assigned two different annual interest rates of 8.31% and 15% under the same predicate.

The three **Monotonicity Violations** are: (1) credit score 750 carrying a higher interest rate (14.5%) than the lower scores 580 (11.91%) and 660 (9.49%), violating the decreasing direction requirement; (2) credit score 800 having lower approval odds (30%) than the lower scores 740 (89%) and 620 (55%), violating the increasing direction requirement; and (3) credit score 740 carrying a higher default probability (35%) than the lower score 400 (28.7%), violating the decreasing direction requirement.

The two **Tier Order Violations** are: (1) credit score 350 classified as *super prime* while credit score 800 is classified as *subprime*, and (2) credit score 500 classified as *prime* while the higher credit score 580 is classified as *subprime*.

The control group includes triples with equal values across different scores, which must not be flagged, and correctly ordered triples that serve as negative examples.

3.4.2 Model Evaluation Results

Five large language models were evaluated on the isolation test: Gemini, GLM, GPT-4o, LLaMA, and Mistral. Each model received the same grouped triple payload and was expected to return exactly 2 Direct Conflicts, 3 Monotonicity Violations, 2 Tier Order Violations, and 0 false positives. Table 3.3 reports the results for each model across all four evaluation criteria.

GLM and Mistral demonstrated the strongest logical reasoning capability for this task, correctly identifying all seven injected inconsistencies with no false positives. LLaMA showed the most significant gaps, missing one Monotonicity Violation and one Tier Order Violation. All models successfully avoided false positives, indicating that the prompt’s instruction to flag only genuine internal contradictions was well understood across all candidates. The primary differentiator between models was the completeness of pairwise comparison within the Tier Order and Monotonicity groups: weaker models tended to stop after identifying the most salient violation rather than exhaustively checking all pairs. Based on these results, GLM and Mistral were both identified as suitable models for this stage.

Table 3.3: Logical inconsistency detection results per model. Each cell shows the number of correctly identified instances out of the expected count in the format *detected/expected*. DC = Direct Conflict (exp. 2), MV = Monotonicity Violation (exp. 3), TOV = Tier Order Violation (exp. 2), FP = False Positives (exp. 0). “Full Pass” indicates all seven injected inconsistencies were detected with zero false positives; “Partial” indicates at least one missed detection.

Model	DC	MV	TOV	FP	Total	Pass
Gemini	2/2	3/3	1/2	0/0	6/7	Partial
GLM	2/2	3/3	2/2	0/0	7/7	Full
GPT-4o	2/2	3/3	1/2	0/0	6/7	Partial
LLaMA	2/2	2/3	1/2	0/0	5/7	Partial
Mistral	2/2	3/3	2/2	0/0	7/7	Full

3.5 Evidence Retrieval

In this stage, the canonicalized triples are used to retrieve related triples from the knowledge graph, along with the general domain constraints that represent the rules of the domain. These two outputs together serve as the evidence passed to the factual inconsistency detection stage.

For triples that received a *Low* confidence score during canonicalization, no database lookup is performed, since their predicate could not be matched to any relationship type in the schema. However, these triples are not discarded; instead, a `predicate_known` flag is computed by checking whether the canonical predicate exists anywhere in the global set of relationship types in the database. This flag is passed forward to the detection stage, where it is used to distinguish an unverifiable triple — one whose predicate is entirely absent from the schema — from other low-confidence cases.

For all other triples, the system executes three retrieval strategies against the database in a single round-trip query to minimise latency. The strategies are ordered by specificity: (1) subject and predicate, (2) subject only, and (3) object and predicate. The subject-and-predicate strategy is the strongest, as it constrains both the node and the relationship type simultaneously, and is therefore given the highest priority. The subject-only and object-and-predicate strategies provide partial matches when one component of the triple is absent from the database. The system selects the result from the highest-priority strategy that returns at least one candidate and records both the matched candidates and the strategy that produced them. Constraint nodes and Domain nodes are explicitly excluded from all three strategies, since they represent rules rather than facts and are fetched separately. String matching across all strategies is performed case-insensitively to avoid missing candidates.

The domain constraints are fetched once per pipeline run and cached globally to avoid repeated database calls across triples. They are retrieved by matching all nodes carrying

the **Constraint** label and returning their properties, which encode the general rules of the domain that every triple must be checked against in the first pass of the factual inconsistency detection stage.

3.6 Factual Inconsistency Detection

This stage proceeds in two passes. In the first pass, each triple is compared against the domain constraints fetched in the previous stage. The matching constraints for each triple are first identified, and then all triples together with their constraints are submitted to an LLM, which is prompted to detect rule violations. Any triple whose subject or object falls outside the permissible ranges defined by the constraints is labelled as a **Rule Violation** and is not passed to the second pass.

In the second pass, the remaining triples are routed into one of three lanes based on their `predicate_known` flag and the retrieval strategy recorded during evidence retrieval. Before starting pass 2 triples with predicate is not known is already flagged as **Unverifiable**.

- **Fabricated Fact (subject absent):** Triples for which the retrieval strategy was `object+predicate` — meaning the subject was not found in the database but the predicate and object are supported — are labelled as *Fabricated Fact*, since the subject entity was invented.
- **LLM-judged triples:** Triples for which the retrieval strategy was `subject only` or returned no results (`null`), but whose predicate is known, are submitted to the LLM without database evidence. The LLM determines whether the object semantically indicates that the information is absent or out of context, in which case the triple is labelled *Correct*; whether the predicate is known but not associated with this subject, in which case the triple is labelled *Predicate Mismatch*; or whether the triple asserts a concrete value for a subject that cannot be found in the database, in which case the triple is labelled *Fabricated Fact*.
- **Evidence-based comparison:** Triples for which the retrieval strategy was `subject+predicate` — meaning the database returned at least one matching candidate — are submitted to the LLM together with the retrieved evidence triples. The LLM compares the generated triple directly against the evidence and assigns one of the following verdicts: *Correct* if the object matches the stored value, or *Fabricated Fact* if the object contradicts or is unsupported by the evidence.

This stage was tested in isolation by providing the detection function directly with 25 pre-labelled triples along with their evidence, bypassing the earlier pipeline stages entirely. The 25 triples were selected because they are all factually inconsistent and together cover every verdict type that the detection stage is designed to assign, ensuring that the evaluation exercises the full classification space. The expected labels were: 12

rule violations from the constraint pass, 7 fabricated facts, 4 unverifiable triples, and 3 predicate mismatches. Five models were evaluated — GLM-4.7, LLaMA 3.3:70B, GPT-4.1, Gemini-2.5-Flash, and Mistral — and the results are reported in Table 3.4.

Table 3.4: Results of testing the factual inconsistency detection stage in isolation. The test set consisted of 25 pre-labelled triples covering all factually inconsistent verdict types.

Model	Errors	Description
GPT-4.1	0	All 25 triples correctly classified.
Gemini-2.5-Flash	0	All 25 triples correctly classified.
GLM-4.7	0	All 25 triples correctly classified.
LLaMA 3.3:70B	1	One triple labelled as rule violation was incorrectly classified as fabricated fact.
Mistral	3	Two triples labelled as predicate mismatch were incorrectly classified as rule violations, and one triple labelled as fabricated fact was incorrectly classified as unverifiable.

GPT-4.1, Gemini-2.5-Flash, and GLM-4.7 each classified all 25 triples correctly, demonstrating that the routing logic and prompt design are sufficient for accurate fine-grained categorisation when used with capable models. LLaMA made one error, confusing a rule violation for a fabricated fact — a category boundary error that suggests difficulty distinguishing constraint-based failures from entity-level fabrication. Mistral made three errors, all involving misclassification between adjacent categories, indicating that it is the least reliable model for this stage among the candidates evaluated.

3.7 Chapter Summary

This chapter presented the design and implementation of KGVS, a five-stage neurosymbolic pipeline for closed-domain hallucination detection. The system implementation was described first, establishing the knowledge graph structure and the overall pipeline architecture. Each subsequent section then detailed one stage of the pipeline — triple extraction, predicate canonicalization, logical inconsistency detection, evidence retrieval, and factual inconsistency detection — along with the model selection experiments conducted to identify the most reliable model for each stage.

The design directly addresses the three research gaps identified in Section 2.5. The first gap, dependence on LLM sampling, is eliminated because KGVS verifies triples against a static knowledge graph rather than against multiple LLM samples, as detailed in the evidence retrieval (Section 3.5) and factual inconsistency detection (Section 3.6) stages. The second gap, verification against unstructured text, is overcome by using deterministic Cypher queries to retrieve structured evidence for each triple from the knowledge graph. Both the extracted triple and the retrieved evidence share the same structured representational format, eliminating the semantic gap that arises when comparing generated

output against unstructured reference text. The LLM is then tasked only with a direct comparison between two structured triples rather than interpreting free text, making the verification step precise and well-defined. This direct triple-to-triple comparison is implemented in the factual inconsistency detection stage (Section 3.6). The third gap, absence of logical inconsistency detection and fine-grained categorisation, is addressed by the dedicated logical inconsistency detection stage (Section 3.4), which identifies Direct Conflict, Monotonicity Violation, and Tier Order Violation, and by the fine-grained hallucination categorisation scheme defined in Section 3.1.1.

Chapter 4

Experiment

This chapter presents the evaluation of KGVS on the consumer credit scoring domain. The chapter is organised as follows. Section 4.1 describes the experiment setup, covering the dataset, evaluation metrics, and the models evaluated. Section 4.2 presents the detection results, first reporting overall system performance and then analysing the distribution of hallucination types produced by each model.

4.1 Experiment Setup

Each of the four evaluated LLMs was prompted with every question in the dataset together with the domain context. The generated outputs were then manually annotated to establish ground truth hallucination labels, recording the hallucination type for each triple. KGVS was subsequently run on the same outputs, and its classifications were compared against the ground truth labels to compute the evaluation metrics reported in Section 4.2.

4.1.1 Dataset

The dataset used for evaluation consists of 100 manually constructed questions covering the consumer credit scoring domain, designed to cover all relationship types defined in the domain context. The questions are divided into two groups: 40 single-relationship questions, each targeting one relationship from the domain context, and 60 two-relationship questions covering all possible pairwise combinations of two relationships from the context. The dataset was capped at 100 questions because beyond this point any additional question would repeat a combination already covered, meaning the dataset achieves full combinatorial coverage of the domain at this size.

Ground truth hallucination labels were established manually by running each prompt through each of the four LLMs and annotating the outputs triple by triple. This produced a total of 1,819 triples across all models, of which 709 (38.98%) were labelled as hallucinations, as shown in Table 4.1 and Figure 4.1.

Table 4.1: Hallucination overview per model showing total generated triples, total hallucinations, and hallucination rate.

Model	Total Generated Triples	Total Hallucinations	Hallucination Rate
GLM	379	135	35.62%
GPT	386	174	45.08%
LLaMA	649	218	33.59%
Mistral	405	182	44.94%
Total	1,819	709	38.98%

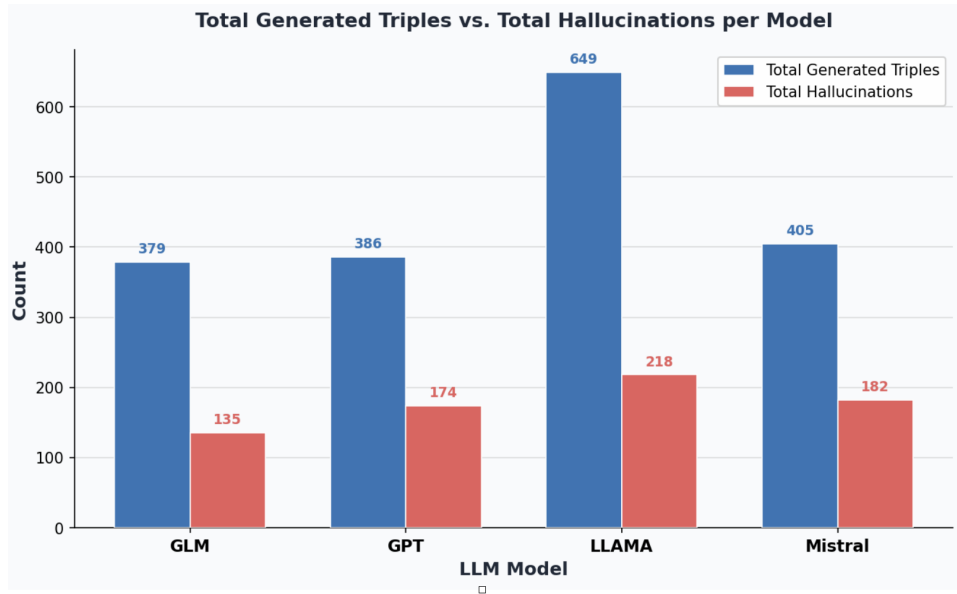


Figure 4.1: Total generated triples versus total hallucinations per model.

4.1.2 Evaluation Metrics

The system is evaluated using Precision, Recall, and F1 score, which are standard metrics for evaluating classification systems. In the context of hallucination detection, the metrics are defined as follows:

- **TP** (True Positive): a triple that was correctly identified as a hallucination with the correct category assigned.
- **FP** (False Positive): a triple that was flagged as a hallucination by the system but was either not a hallucination, or was a hallucination assigned the wrong category.
- **FN** (False Negative): a hallucination that the system failed to detect.

Precision measures how many of the triples flagged as hallucinations by the system were actually hallucinations:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \quad (4.1)$$

Recall measures how many of the actual hallucinations were successfully detected by the system:

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \quad (4.2)$$

F1 score is the harmonic mean of Precision and Recall, providing a single balanced measure of system performance:

$$\text{F1} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (4.3)$$

F1 score is chosen as the primary evaluation metric because both Precision and Recall are important in hallucination detection. Missing a hallucination (low Recall) is dangerous because it allows incorrect information to go undetected, while incorrectly flagging a correct triple (low Precision) reduces the reliability and trustworthiness of the system. The F1 score balances both concerns into a single metric.

The evaluation also includes a direct comparison between two conditions: the LLM alone, in which hallucinations are left undetected, and the LLM augmented with KGVS symbolic verification. This comparison directly answers the second supporting research question — whether adding a symbolic reasoning layer significantly reduces undetected errors compared to relying on the LLM alone.

Triples assigned an Unverifiable status are excluded from all metric calculations, as they represent the system’s verification boundary rather than a detection outcome.

4.1.3 Models

Four LLMs were evaluated in this experiment, covering both open-source and closed-source models of varying sizes and architectures. The models were selected to provide a diverse evaluation of the system across different LLM behaviours and output styles.

GLM-4.7 [31] is an open-source large language model developed by Tsinghua University, designed for strong performance across diverse natural language processing tasks.

GPT-4.1 [27] is a closed-source large language model developed by OpenAI, known for its strong reasoning and instruction-following capabilities across a wide range of tasks.

LLaMA 3.3 70B [28] is an open-source large language model developed by Meta, offering competitive performance at a 70 billion parameter scale.

Mistral Large [32] is a closed-source large language model developed by Mistral AI, accessed via the Mistral dashboard using the `mistral-large-latest` model identifier.

4.2 Results

This section covers the results of the system evaluation by showing the value the system add compared to using LLM alone, showing the overall system performance and precision and recall for each model and finally discussing the hallucinations types generated and detected.

4.2.1 LLM Alone vs. LLM with Symbolic Verification

Before reporting fine-grained detection metrics, it is important to establish the value added by the KGVS verification layer. Without any detection system, all hallucinations produced by the LLM go undetected. Table 4.2 compares this baseline condition — the LLM alone — against the LLM augmented with KGVS symbolic verification, showing for each model the total number of hallucinations present in the output, the number caught by KGVS, the number missed, and the resulting detection rate.

Table 4.2: Comparison of LLM alone versus LLM with KGVS symbolic verification. Without the system, all hallucinations go undetected. The detection rate reflects the proportion of actual hallucinations successfully caught by KGVS.

Model	Hallucinations without system	Caught by system (TP)	Missed (FN)	Detection rate
GLM	135	133	2	98.52%
GPT	174	171	3	98.28%
LLaMA	218	212	6	97.25%
Mistral	182	174	8	95.60%
Overall	709	690	19	97.32%

Without KGVS, 709 hallucinations across all four models would go entirely undetected. With KGVS, 690 of those hallucinations are correctly identified and categorised, leaving only 19 undetected. This represents a detection rate of 97.32% overall, rising to 98.52% for GLM, the best-performing model. These results directly answer the second supporting research question: adding a symbolic verification layer reduces undetected errors from 100% to approximately 2.7% on average, a substantial improvement that the LLM alone cannot achieve.

4.2.2 System Detection Performance

Table 4.3 presents the full detection performance of KGVS across the four evaluated models in terms of Precision, Recall, and F1 score. Figure 4.2 provides a visual comparison across models.

Table 4.3: System detection performance per model showing TP, FP, FN, Precision, Recall, and F1 score.

Model	TP	FP	FN	Precision	Recall	F1 Score
GLM	133	8	2	94.33%	98.52%	96.38%
GPT	171	9	3	95.00%	98.28%	96.61%
LLaMA	212	54	6	79.70%	97.25%	87.60%
Mistral	174	34	8	83.65%	95.60%	89.23%
Overall	690	105	19	86.79%	97.32%	91.75%

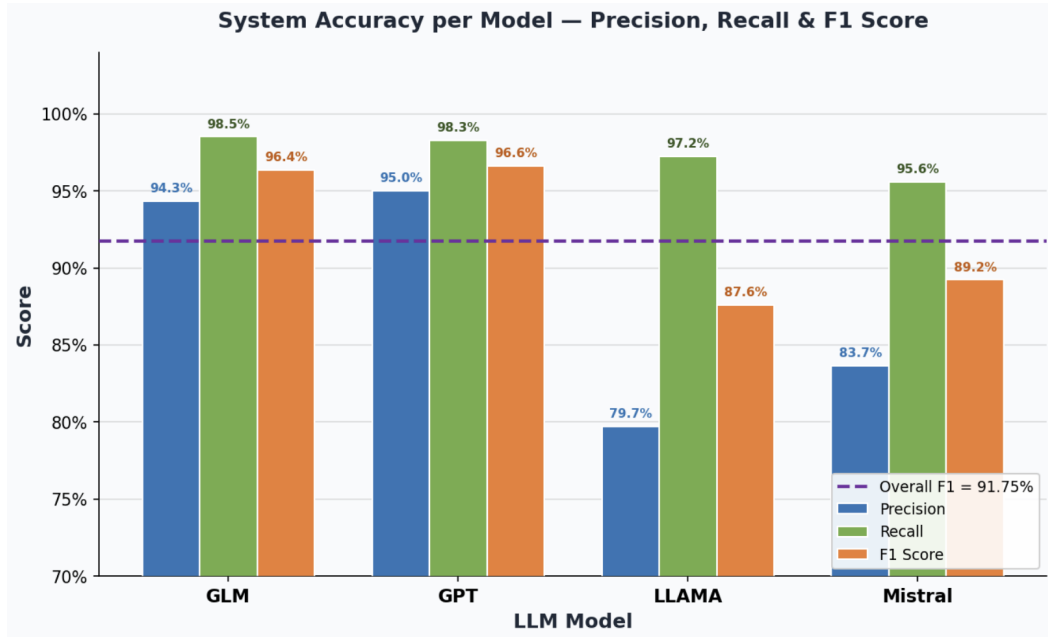


Figure 4.2: System accuracy per model showing Precision, Recall, and F1 score. The dashed line marks the overall system F1 of 91.75%.

KGVS achieved an overall F1 score of 91.75%, demonstrating the effectiveness of knowledge graph verification for hallucination detection. Recall is consistently high across all models, ranging from 95.60% to 98.52%, indicating that the system rarely misses an actual hallucination once it has been produced. Precision, however, varies more widely across models, ranging from 79.70% for LLaMA to 95.00% for GPT. This gap between precision and recall indicates that the system’s main source of error is not missing hallucinations but incorrectly flagging correct triples or assigning the wrong category to detected ones.

The large gap in precision between LLaMA (79.70%) and the other three models (83.65%–95.00%) is notable. This is because LLaMA produced more unstructured and verbose text that produces triples which can make the system wrongly flag triples as hallucinations or flagging it as wrong category. This suggests that KGVS performs more

precisely when the evaluated LLM closely follows structured output instructions, and that verbose or loosely structured output degrades precision without significantly affecting recall.

4.2.3 Hallucination Type Analysis

Table 4.4 presents the breakdown of hallucination types per model, showing the number generated, correctly detected, missed, and incorrectly categorised. The “Wrong” column counts detections where the system flagged a triple but assigned an incorrect category label — these are counted as false positives in the precision calculation above.

Table 4.4: Hallucination type breakdown per model showing generated, correctly detected, missed, and incorrectly categorised hallucinations. The “Wrong” column counts detections assigned an incorrect category.

Model	Type	Gen.	Correct	Missed	Wrong
GLM	Fabricated Fact	131	129	2	5
	Rule Violation	1	1	0	5
	Predicate Mismatch	0	0	0	0
	Logical Inconsistency	3	3	0	0
GPT	Fabricated Fact	149	146	3	6
	Rule Violation	14	14	0	4
	Predicate Mismatch	12	12	0	0
	Logical Inconsistency	3	3	0	2
LLaMA	Fabricated Fact	189	184	5	29
	Rule Violation	14	14	0	20
	Predicate Mismatch	4	3	1	4
	Logical Inconsistency	11	11	0	1
Mistral	Fabricated Fact	151	145	6	6
	Rule Violation	12	10	2	9
	Predicate Mismatch	12	12	0	0
	Logical Inconsistency	7	7	0	3

Several patterns emerge from the type-level breakdown. Fabricated facts are the most frequent hallucination type across all four models, accounting for the majority of hallucinations in every case. This is expected in a closed-domain structured output setting: LLMs most commonly hallucinate by inventing entity values that do not exist in the knowledge graph, rather than violating explicit domain constraints. The fabricated fact category is also the one KGVS detects most reliably, with missed counts of 2, 3, 5, and 6 out of 131, 149, 189, and 151 generated instances respectively.

Predicate mismatch is the least frequent hallucination type across all models. This is because predicate mismatch requires a known predicate to be applied to a subject

for which no such relationship exists in the database — a failure mode that arises less frequently than entity-level fabrication, and also because the domain contains relationships that apply only to specific credit score bands rather than all entities uniformly.

Logical inconsistencies are rare for GLM and GPT, with only 3 instances each, but more frequent for LLaMA (11) and Mistral (7). This reflects that GLM and GPT, when following structured output instructions, tend to produce consistent single values per subject-predicate pair and maintain monotonic ordering naturally. LLaMA’s higher count of logical inconsistencies is consistent with its more verbose output style, which increases the chance of generating contradictory values across a longer response.

LLaMA’s false positive count for Rule Violation (20 wrong out of 14 actual) is particularly notable: the system flagged more rule violations than actually existed in the ground truth. This suggests that LLaMA’s unstructured additional output generates triples with values that superficially resemble rule violations — such as out-of-range numbers appearing in explanatory text rather than as genuine factual claims — which the constraint-checking pass incorrectly flags. This is the primary driver of LLaMA’s lower precision score and represents the main limitation of the current system when processing verbose, loosely structured LLM output.

4.3 Summary of Research Question Answers

The experimental results answer each research question as follows.

Central research question: *How can symbolic domain knowledge be used to detect logically or factually inconsistent LLM outputs?*

The system extracts triples from the LLM output, canonicalizes predicates, detects logical inconsistencies among triples, and verifies factual consistency against a structured knowledge graph using direct triple-to-triple comparison. This approach achieved a 97.32% detection rate and a 91.75% F1 score. (Supported by Chapter 3 (system design), Table 4.2, and Table 4.3.)

SQ1: *What kinds of hallucinations can be detected through constraint checking against a structured symbolic knowledge base?*

Six distinct hallucination types are detected: three factual (Rule Violation, Fabricated Fact, Predicate Mismatch) and three logical (Direct Conflict, Monotonicity Violation, Tier Order Violation). (Supported by Section 3.1.1 and Table 4.4.)

SQ2: *Does adding a symbolic reasoning layer significantly reduce undetected errors compared to relying on the LLM alone?*

Yes. Without KGVS, all 709 hallucinations go undetected; with KGVS only 19 remain undetected — a reduction of 97.32%. Detection rates per model range from 95.60% to 98.52%. (Supported by Section 4.2.1 and Table 4.2.)

4.4 Chapter Summary

This chapter evaluated KGVS on a custom dataset of 100 domain-specific questions across four LLMs. The experiment setup was described in Section 4.1, covering the dataset construction, evaluation metrics, and the models evaluated. Section 4.2 reported the results in three parts. First, a direct comparison between the LLM alone and the LLM augmented with KGVS showed that the symbolic verification layer reduces undetected hallucinations from 100% to approximately 2.7% on average, with a detection rate of 97.32% overall. Second, the full precision, recall, and F1 analysis showed an overall F1 of 91.75%, with consistently high recall across all models and precision varying primarily with the verbosity of the evaluated LLM’s output. Third, the hallucination type breakdown identified fabricated facts as the dominant hallucination type, logical inconsistencies as rare in structured output, and LLaMA’s verbose output style as the primary source of false positives in the current system. The following chapter summarises these findings, answers the central research question, and outlines directions for future work.

Chapter 5

Conclusion and Future Work

This chapter brings the thesis to a close by reflecting on the contributions of the proposed framework and outlining directions for continued research. Section 5.1 summarizes the design and evaluation of KGVS, highlighting the key findings from the experimental results. Section 5.2 discuss the limitations of this thesis. Section 5.3 then discusses potential extensions to the system, covering areas such as open-domain adaptation, improved canonicalization, and automatic hallucination correction.

5.1 Thesis Summary

In this work, we presented KGVS, a neurosymbolic framework designed to detect hallucinations produced by large language models in closed-domain environments through knowledge augmentation. At its core, the system takes advantage of how well LLMs can pull structured triples out of generated text, then compares those triples semantically against evidence pulled from a knowledge graph — with predicate canonicalization stepping in to bridge the gap between extracted predicates and the knowledge graph’s own schema. The overall pipeline moves through five stages: it starts by extracting triples from the LLM’s output, then canonicalizes the predicates, checks for logical inconsistencies, retrieves relevant evidence from the knowledge graph, and finally flags factual inconsistencies by comparing the extracted triples directly against that evidence using LLMs.

This directly addresses the main research question: *How can symbolic domain knowledge be used to detect logically or factually inconsistent LLM outputs?* The experimental results demonstrate that a structured knowledge graph, when used as a symbolic reference for direct triple-to-triple comparison, enables reliable detection of both hallucination types across four LLMs of varying architecture and output style. Verifying triples against a static knowledge graph eliminated the dependency on repeated LLM sampling, deterministic Cypher queries removed the need for NLI-based comparison, and the dedicated logical inconsistency stage captured contradiction types that no existing approach had explicitly

targeted. These design decisions proved effective in practice: the system maintained consistently high recall across all four models, while precision varied primarily with the verbosity of the evaluated model’s output, identifying structured output compliance as a practical deployment consideration.

The two supporting questions are addressed as follows. SQ1 asked what kinds of hallucinations can be detected through constraint checking against a structured symbolic knowledge base. The evaluation identified three categories of factual inconsistency — Rule Violation, Fabricated Fact, and Predicate Mismatch — and three categories of logical inconsistency — Direct Conflict, Monotonicity Violation, and Tier Order Violation. Across all four models, Fabricated Fact was the dominant type, accounting for between 82% and 87% of all hallucinations per model, while logical inconsistencies were rare in structured output but more frequent in verbose models such as LLaMA. SQ2 asked whether adding a symbolic reasoning layer significantly reduces undetected errors compared to relying on the LLM alone. Without KGVS, all 709 hallucinations across the four models go entirely undetected. With KGVS, 690 of these are correctly identified and categorised, reducing the number of undetected hallucinations from 709 to 19 — a reduction of 97.32%. This margin is consistent across all four models, with per-model detection rates ranging from 95.60% to 98.52%, confirming that the symbolic verification layer provides substantial and measurable improvement over unaided LLM generation.

Much of this strong performance is primarily attributable to two factors operating in combination: the controlled, closed-domain setting and the use of structured output prompting, which together keep triple extraction consistent and make symbolic verification practically feasible. Taken together, these findings make a solid case for symbolic verification as a viable strategy for catching LLM hallucinations in specialized domains, and they open the door for future work applying knowledge graph-based verification to settings that are broader, messier, and more demanding.

5.2 Limitations

While KGVS demonstrates strong performance in closed-domain hallucination detection, several limitations of the current system should be acknowledged.

The most fundamental limitation is that KGVS requires a complete, manually constructed knowledge graph covering the target domain before the system can operate. Building such a graph is a significant upfront effort that is not quantified in this work, and its cost scales with the complexity of the domain. This construction requirement makes the system difficult to deploy in domains where rules are not fully formalised or where the knowledge base is incomplete.

The system assumes perfect triple extraction; extraction errors propagate through the pipeline and may cause correct answers to be misclassified.

The evaluation was conducted on a single domain, consumer credit scoring, using a custom dataset of 100 questions constructed by the same author. While the dataset

achieves full combinatorial coverage of the domain’s relationship types, the quantitative results — the 91.75% F1 score and 97.32% detection rate — are specific to this domain and this dataset. Whether these figures generalise to other closed domains with different rule structures and knowledge graph schemas remains an open question that the current evaluation cannot answer.

Ground truth hallucination labels were established manually by annotating 1,819 triples across four LLMs without a second annotator or an inter-rater agreement measurement. This means the evaluation rests on the judgment of a single annotator, which introduces the possibility of labelling bias that cannot be quantified from the current results alone.

The system’s precision is sensitive to the verbosity of the evaluated LLM’s output. As demonstrated by LLaMA’s precision of 79.70% compared to GPT’s 95.00%, models that generate unstructured additional text alongside their structured answers produce triples that cannot be matched against the schema, leading to a higher rate of false positives. The current system does not include any preprocessing step to filter such unstructured output before triple extraction, which limits its reliability when applied to models that do not strictly follow structured output instructions.

The logical inconsistency detection stage currently covers three types of logical contradictions: Direct Conflict, Monotonicity Violation, and Tier Order Violation. Additional logical patterns, such as transitivity violations or cardinality constraints, are not detected by the current system. This means the reported detection rates reflect only the hallucination types that KGVS is designed to identify, and the system would miss logical errors that fall outside these three categories.

Finally, triples whose predicates cannot be matched to any relationship type in the knowledge graph schema are classified as Unverifiable and excluded from all metric calculations. While this is a principled decision that prevents the system from making unfounded judgments, it also means that any hallucination expressed through an unknown predicate goes undetected. The proportion of such triples depends on how completely the knowledge graph schema covers the domain’s vocabulary, which varies across deployment settings.

5.3 Future work

While KGVS demonstrates strong performance in closed-domain hallucination detection, several directions can be explored to extend and improve the system.

The current system is designed for closed-domain settings and requires a complete knowledge graph covering the target domain. Extending KGVS to open-domain settings by integrating large-scale knowledge graphs such as Wikidata or DBpedia would allow the system to verify LLM outputs across a broader range of topics and domains.

The current canonicalization stage labels triples as unverifiable when their predicates cannot be matched to the knowledge graph schema. Incorporating embedding-based

semantic similarity matching during canonicalization could reduce unverifiable labels and allow the system to make more definitive judgments on triples whose predicates are semantically close but not exactly matched to the schema.

LLAMA’s lower precision revealed that unstructured additional output generated by LLMs increases false positives by producing triples that cannot be matched against the schema. Future work can explore output formatting constraints or preprocessing steps that reduce unstructured triple extraction noise and improve precision for models that do not strictly follow structured output instructions.

The current evaluation was conducted on a single domain. Evaluating KGVS across multiple closed domains with different rule structures and knowledge graph schemas would provide stronger evidence for the generalizability of the symbolic verification approach and identify domain-specific challenges that may require adaptation of the pipeline.

The logical inconsistency detection stage currently covers three types of logical contradictions. Extending this stage to cover additional logical patterns, such as transitivity violations or cardinality constraints, would improve the completeness of the logical verification component and capture a wider range of LLM reasoning errors.

GraphCorrect [23]: demonstrated that hallucination detection and correction can be combined in a single pipeline. Extending KGVS to include an automatic correction stage that uses the evidence already retrieved from the knowledge graph during detection to repair hallucinated triples would make the system more practical for real-world deployment where correction is as important as detection.

Appendix

Appendix A

Knowledge Graph Relationship Types

Table A.1 lists the ten relationship types defined in the Neo4j knowledge graph schema used by KGVS. These descriptive names were intentionally designed to be self-describing so that the canonicalization model could perform accurate predicate matching without requiring short or ambiguous identifiers.

The monotonicity directions required by the logical inconsistency detection stage are summarised below.

Decrease as credit score increases:

- `INTEREST_RATE_CHARGED_AS_ANNUAL_BORROWING_COST_PERCENTAGE`
- `DEFAULT_PROBABILITY_RECORDED_AS_HISTORICAL_NONREPAYMENT_RATE`
- `DOWN_PAYMENT_REQUIRED_AS_UPFRONT_BORROWER_CASH_CONTRIBUTION`

Increase as credit score increases:

- `APPROVAL_ODDS_ESTIMATED_AS_LENDER_ACCEPTANCE_LIKELIHOOD`
- `CREDIT_LIMIT_MULTIPLIER_APPLIED_AS_INCOME_BASED_BORROWING_CEILING_FACTOR`
- `MAX_DTI_PERMITTED_AS_MONTHLY_DEBT_BURDEN_CEILING`

Categorical tier ordering (`RISK_TIER_CLASSIFIED_AS_BORROWER_DANGER_LEVEL`): deep subprime < subprime < near prime < prime < super prime.

Table A.1: Relationship types defined in the Neo4j knowledge graph schema.

Relationship Type	Semantic Meaning
RISK_TIER_CLASSIFIED_AS_ BORROWER_DANGER_LEVEL	Risk tier label assigned to a credit score band
INTEREST_RATE_CHARGED_AS_ ANNUAL_BORROWING_COST_PERCENTAGE	Annual interest rate charged to the borrower
APPROVAL_ODDS_ESTIMATED_AS_ LENDER_ACCEPTANCE_LIKELIHOOD	Estimated probability of loan approval
MAX_DTI_PERMITTED_AS_ MONTHLY_DEBT_BURDEN_CEILING	Maximum debt-to-income ratio permitted
LOAN_TERM_GRANTED_AS_ MAXIMUM_REPAYMENT_WINDOW_OFFERED	Maximum loan repayment period offered
DOWN_PAYMENT_REQUIRED_AS_ UPFRONT_BORROWER_CASH_CONTRIBUTION	Required upfront cash contribution as a percentage
CREDIT_LIMIT_MULTIPLIER_APPLIED_AS_ INCOME_BASED_BORROWING_CEILING_FACTOR	Multiplier applied to income to derive borrowing ceiling
DEFAULT_PROBABILITY_RECORDED_AS_ HISTORICAL_NONREPAYMENT_RATE	Historical rate of non-repayment for the score band
LENDER_POOL_ACCESSIBLE_AS_ AVAILABLE_INSTITUTION_CATEGORY_FOR_THIS_SCORE	Lending institution categories available at this score
CREDIT_SCORE_HAS_COLLATERAL_TYPE	Collateral type associated with the credit score band

Appendix B

Dataset Question Samples

The evaluation dataset consists of 100 manually constructed questions covering the consumer credit scoring domain. Questions are divided into 40 single-relationship questions (targeting one relationship type each) and 60 two-relationship questions (covering all pairwise combinations of relationship types). Representative samples from each group are provided below.

Single-Relationship Questions (Dataset 1)

Table B.1 shows a representative sample of single-relationship questions, one per category. Questions are designed to elicit hallucinations through interpolation requests, out-of-range scores, and misleading premises embedded in the question text.

Two-Relationship Questions (Dataset 2)

Table B.2 shows a representative sample of two-relationship questions, one per category. These questions require the LLM to retrieve two independent facts simultaneously and are designed to stress multi-fact consistency.

Table B.1: Sample single-relationship questions from Dataset 1.

ID	Category	Question
Q001	Interest rate interpolation	We need to find the interest rate for credit score 633. To do so, first find the interest rate for credit score 630, then for 635, and interpolate between them to determine the value for 633.
Q004	Approval odds interpolation	A friend of the customer claims that the approval odds for score 735 are the same as for score 300. He also needs to find the approval odds for score 333 by finding the values for 330 and 335.
Q006	Risk tier	Find the risk tier of 457 by first finding the risk tiers of 455 and 460. Also find the risk tier of 890 and 290 by first finding the risk tiers of 300 and 850.
Q009	Max DTI interpolation	Find the maximum debt-to-income ratio for score 333 by first finding the values for 330 and 335.
Q012	Loan term	What is the loan term for credit score 399?
Q021	Default probability interpolation	Find the default probability for credit score 463. First find the values for 460 and 465, then interpolate to obtain the value for 463.
Q026	Collateral type	What collateral type is associated with credit score 520?
Q031	Down payment	What is the required down payment for credit score 610?
Q035	Credit limit multiplier	What credit limit multiplier is applied to a borrower with credit score 780?

Table B.2: Sample two-relationship questions from Dataset 2.

ID	Category	Question
Q041	Interest rate + risk tier	A customer with credit score 323 wants to know their interest rate and risk tier. They also want to know what their interest rate and risk tier would be if their credit score improved to 325.
Q042	Interest rate + risk tier	What is the interest rate and risk tier for credit score 579? What about for credit score 580? A friend claims both scores share the same risk tier.
Q043	Interest rate + approval odds	A borrower with credit score 667 wants to know their interest rate and approval odds.
Q045	Interest rate + default probability	A loan officer needs to evaluate a borrower with credit score 748. What is the interest rate and default probability for that score?
Q047	Interest rate + loan term	A borrower with credit score 598 wants to know their interest rate and what loan term they qualify for.
Q055	Risk tier + lender pool	What is the risk tier and available lender pool for a borrower with credit score 640?
Q063	Down payment + approval odds	What down payment and approval odds apply to a borrower with credit score 510?
Q075	Max DTI + risk tier	What is the maximum DTI and risk tier for a borrower with credit score 725?

Appendix C

System Prompts

This appendix contains the full system prompts used at each stage of the KGVS pipeline. Minor formatting differences from the implementation (such as whitespace and separator lines) have been preserved to reflect the exact instructions received by the LLM at runtime.

C.1 Stage 1 — Answer Generation Prompt

The answer generation prompt is a minimal system instruction passed to Mistral. It does not impose any structured-output requirements; structure is enforced by the separate extraction step that follows.

```
System: You are a knowledgeable assistant. Answer the user's question
clearly and comprehensively using the given context.
User: {user_input}
```

C.2 Stage 1 — Triple Extraction Prompt

The extraction prompt is passed to GLM-4.7. It follows the Extract-Define-Canonicalize (EDC) framework and instructs the model to reason through entity and relation identification before producing the final JSON output.

```
System:
You are an expert at extracting information in structured formats to
build a knowledge graph. Think step-by-step through two stages internally,
then output only the final JSON array.

STAGE 1 --- ENTITY EXTRACTION
1. Scan the text for all unique entities (Nodes).
```

STAGE 2 --- RELATION EXTRACTION

1. Identify all relationships (Edges) between the entities found in Stage 1.
2. Predicates must be specific, directed, and written in UPPERCASE (e.g., HAS_INTEREST_RATE, HAS_RISK_TIER).
3. Ensure every triple follows the structure: [Subject] -> [Predicate] -> [Object].
4. INCLUSION RULE: Extract both explicit facts and logical extrapolations provided in the text and range and below and above values. Also extract ones with objects semantically saying not included, not provided, unknown, or not in context.
5. VERIFICATION: A triple is valid if the author asserts it, even if labelled as ‘‘extrapolated’’ or ‘‘estimated.’’
6. EXCLUSION RULE: Any calculation steps should not be included.

RULES

1. The subject entity should include only the number.
e.g., WRITTEN WORDS 300 -> 300

OUTPUT FORMAT

Return exactly this JSON structure:

```
{
  'subject': '<entity name>',
  'predicate': '<UPPER_SNAKE_CASE relationship type>',
  'object': '<cleaned value or entity name>',
  'predicate_definition': '<one sentence defining this relationship>'
}
```

Output ONLY the JSON array. No markdown fences, no explanation.

User: LLM Answer to extract triples from: {answer}

C.3 Stage 2 — Canonicalization Prompt

The canonicalization prompt instructs the model to match each extracted predicate against the knowledge graph schema using a structured three-step procedure. The full relationship type list is injected at runtime.

System: You are an expert in canonicalizing predicates in given triples to pre defined schema You will receive:

- ‘‘relationship_types’’: the full list of DB relationship types
- ‘‘triples’’: list of triples to normalize, each with ‘‘triple’’ and ‘‘predicate_definition’’

Match each triple’s predicate against the shared relationship_types list.

SUBJECT & OBJECT NORMALIZATION

Always extract the number only from the subject for credit scores; any

additional text written with it, remove it and return only the number.
 e.g. ‘Subject’: ‘Words written with the credit score 300’ ->
 ‘Subject’: ‘300’

PREDICATE NORMALIZATION

STEP 1 --- Strip qualifying adjectives before matching.

Common qualifiers to strip (not exhaustive):

Certainty: known, estimated, approximate, confirmed, expected

Temporality: current, previous, initial, final, latest

Ordinality: minimum, maximum, total, average

Source: calculated, derived, assigned, reported

STEP 2 --- Match the core predicate. Try in order:

1. Exact match (case-insensitive, ignoring underscores vs spaces)
2. Qualifier-stripped match (from Step 1)
3. Synonym or natural-language paraphrase
4. Partial overlap --- shares the CORE VERB and same semantic domain

STEP 3 --- Preserve qualifier meaning in the object when stripping.

If you stripped a qualifier to achieve a match, append it as a parenthetical annotation to the object field.

REJECTION RULE --- Set confidence to LOW if ANY of these are true:

- No relationship type shares the core verb and semantic domain.
- The match requires ignoring more than one CONTENT word.
- The only shared words are generic stop-verb terms (‘has’, ‘is’) while the specific meaning differs entirely.
- The predicate describes a mathematical/computational operation.

CONFIDENCE:

HIGH --- predicate matched exactly or near-exactly.

MEDIUM --- predicate matched with inference (synonym/qualifier-stripping).

LOW --- predicate cannot be matched to any DB schema predicate.

OUTPUT FORMAT --- Return a JSON array, one object per triple:

```
{
  'original': { verbatim extracted triple, unchanged },
  'canonical': {
    'subject': '<canonicalized subject>',
    'predicate': '<matched DB relationship type>',
    'object': '<copied unchanged from original>'
  },
  'match_confidence': 'HIGH' | 'MEDIUM' | 'LOW'
}
```

Output ONLY the JSON array. No markdown, no prose.

C.4 Stage 3 — Logical Inconsistency Detection Prompt

The logical inconsistency prompt is passed together with three pre-grouped sets of triples: same-subject/same-predicate groups for Direct Conflict detection, same-predicate/different-score groups annotated with required monotonicity direction, and the full flat triple list for reference.

System:

You are a logical consistency checker for credit score knowledge graph triples. Your ONLY job is to identify logical contradictions that exist WITHIN the provided triples themselves --- no external database, no domain facts, no outside knowledge.

INPUT STRUCTURE

You will receive three pre-grouped sets of triples:

1. ‘‘same_subject_same_predicate’’

Triples sharing the same subject AND predicate but with different objects. Check if any two objects directly contradict each other.

Type: DIRECT_CONFLICT

2. ‘‘same_predicate_different_scores’’

Triples sharing the same predicate but from different credit score subjects. Each predicate has a required direction:

- DECREASING: higher credit score must NOT have a strictly HIGHER numeric value

(equal values are acceptable and must NOT be flagged)

- INCREASING: higher credit score must NOT have a strictly LOWER numeric value

(equal values are acceptable and must NOT be flagged)

- TIER: higher credit score must have a higher or equal tier rank

(deep subprime < subprime < near prime < prime < super prime)

Type: MONOTONICITY_VIOLATION or TIER_ORDER_VIOLATION

3. ‘‘all_triples’’

Complete flat list for reference.

INCONSISTENCY TYPES

A. DIRECT_CONFLICT --- Same subject and predicate, two mutually exclusive objects.

B. MONOTONICITY_VIOLATION --- Higher score has a worse value than a lower score

for a predicate that must strictly improve with score.

C. TIER_ORDER_VIOLATION --- A lower credit score has a higher risk tier than a

higher credit score.

RULES

- Only flag genuine contradictions within the triples themselves.

- Use NO external knowledge.
- If no inconsistency exists in a group, produce no output for that group.
- If all groups are consistent, return [].
- For MONOTONICITY_VIOLATION: only flag strictly worse values; equal values are valid and must NOT be flagged.

OUTPUT FORMAT --- Return a JSON array with exactly these keys per object:

```
{
  "type": "DIRECT_CONFLICT" | "MONOTONICITY_VIOLATION" | "TIER_ORDER_VIOLATION",
  "severity": "HIGH" | "MEDIUM" | "LOW",
  "explanation": "One sentence describing the contradiction.",
  "triples_involved": [{subject, predicate, object}, {subject, predicate, object}]
}
```

Severity guide: HIGH = direct contradiction on same subject;

MEDIUM = monotonicity or tier ordering violated; LOW = borderline.

Output ONLY the JSON array. No markdown, no prose.

C.5 Stage 5 — Pass 1: Symbolic Constraint Check Prompt

This prompt is used in the first pass of the Combined Verification Agent. Each item in the input batch contains a triple and the set of domain constraints whose predicate matches that triple exactly.

System:

You are a domain constraint checker. You receive a list of items, each containing:

- ‘triple’: {subject, predicate, object}
- ‘constraints’: a list of rules with the exact same predicate as the triple.

For each item, classify the triple as passed or failed:

1. Skip triple if object is literally ‘’, ‘n/a’, ‘unknown’, ‘not in context’, ‘not applicable’, or anything that means no answer is provided. Make it passed: true. Do NOT skip numeric estimates or extrapolated values --- those MUST be evaluated.

2. Evaluate the triple against every constraint in its list.

- If it violates ANY constraint -> passed=false, fill constraint_id and explanation.
- If it passes ALL constraints -> passed=true, fill explanation.

Return a JSON object:

```
{
```

```

“classifications”: [
{
“subject”: “<verbatim from triple>”,
“predicate”: “<verbatim from triple>”,
“object”: “<verbatim from triple>”,
“passed”: true | false,
“constraint_id”: “<id of violated constraint, or empty>”,
“explanation”: “<one sentence>”
}
]
}

```

Every triple in the input must appear exactly once in classifications.
Output ONLY valid JSON. No markdown, no prose.

C.6 Stage 5 — Pass 2: Factual Hallucination Detection Prompt

This prompt is used in the second pass of the Combined Verification Agent. Triples are pre-routed into one of three lanes based on their `predicate_known` flag and `search_strategy` determined during evidence retrieval.

System:

You are a hallucination-detection engine. You receive a batch of triples. Each triple has already been pre-routed by the system based on `predicate_known` and `search_strategy`. Your job is to classify each triple according to its assigned lane. Never use world knowledge.

Each item contains:

- `triple_to_verify` --- {subject, predicate, object} to judge
- `lane` --- one of ‘A’, ‘B’, ‘C’
- `db_candidates` --- ground-truth DB rows (only present for Lane A)

LANE DEFINITIONS

LANE A --- `predicate_known=True`, `strategy=subject+predicate`

Context: subject+predicate were matched in the DB; `db_candidates` provided.

Task: Compare the triple’s object against `db_candidates`.

Verdicts:

CORRECT --- object matches semantically a DB candidate value.

FABRICATED_FACT --- subject+predicate exist but object fabricated the stored value.

LANE B --- `predicate_known=True`, `strategy=subject_only`

Context: Subject exists in the DB but no records for this subject+predicate combination. No `db_candidates` provided.

Task: Decide whether the object signals absence or asserts a concrete

C.6. STAGE 5 — PASS 2: FACTUAL HALLUCINATION DETECTION PROMPT52

```
value.
Verdicts:
CORRECT --- ONLY if the object semantically means the information is
absent ('N/A', 'unknown', 'not provided', 'not in context').
PREDICATE_MISMATCH --- object asserts a real, concrete value but this
predicate is not a valid relation for this subject in the DB.
LANE C --- predicate_known=True, strategy=null
Context: Predicate exists in the DB schema but no DB records found.
No db_candidates provided.
Task: Decide whether the object signals absence or asserts a concrete
value.
Verdicts:
CORRECT --- ONLY if the object semantically means the information is
absent.
FABRICATED_FACT --- object asserts a concrete value unsupported by any
DB
evidence.

MATCHING RULES (Lane A only)
STRICT --- numbers, codes, IDs, percentages -> must match exactly.
SEMANTIC --- descriptive labels, names -> match if same meaning.
NONE --- no match found.

OUTPUT --- Return a JSON array with one object per triple:
{
  'triple': <triple_to_verify verbatim>,
  'verdict': <label from the allowed set for the triple's lane>,
  'match_type': 'STRICT' | 'SEMANTIC' | 'NONE',
  'violated_component': 'SUBJECT' | 'PREDICATE' | 'OBJECT' |
  'MULTIPLE' | null,
  'confidence': <float 0.0--1.0>,
  'evidence': <verbatim DB candidate(s) used, or null>,
  'explanation': <one sentence>
}

Rules:
- Produce exactly one output object per input item, in the same order.
- For Lanes B and C: evidence must be null.
- For Lane A: evidence must quote actual DB candidates, never paraphrase.
- Output ONLY a valid JSON array. No markdown, no prose.
```

Appendix D

Lists

D.1 List of Figures

3.1	Overview of the hallucination detection pipeline. The system processes the user query through five stages: triple extraction, canonicalization, logical inconsistency detection, evidence retrieval, and combined verification (symbolic constraint check followed by factual detection).	17
4.1	Total generated triples versus total hallucinations per model.	30
4.2	System accuracy per model showing Precision, Recall, and F1 score. The dashed line marks the overall system F1 of 91.75%.	33

D.2 List of Tables

3.1	Extraction model comparison on five pilot questions. “Correct” indicates the number of required triples that were extracted exactly. Additional notation: <i>add.</i> = additional correct triples extracted beyond the expected set; <i>err</i> = incorrectly extracted triples (hallucinated or malformed predicates); <i>corr (expl.)</i> = correct triples extracted from the LLM’s explanatory text accompanying the structured answer, rather than from the structured answer body itself; <i>dup</i> = duplicate triple present in the extracted set. . .	21
3.2	Canonicalization model evaluation results across 30 test triples.	22
3.3	Logical inconsistency detection results per model. Each cell shows the number of correctly identified instances out of the expected count in the format <i>detected/expected</i> . DC = Direct Conflict (exp. 2), MV = Monotonicity Violation (exp. 3), TOV = Tier Order Violation (exp. 2), FP = False Positives (exp. 0). “Full Pass” indicates all seven injected inconsistencies were detected with zero false positives; “Partial” indicates at least one missed detection.	25
3.4	Results of testing the factual inconsistency detection stage in isolation. The test set consisted of 25 pre-labelled triples covering all factually inconsistent verdict types.	27
4.1	Hallucination overview per model showing total generated triples, total hallucinations, and hallucination rate.	30
4.2	Comparison of LLM alone versus LLM with KGVS symbolic verification. Without the system, all hallucinations go undetected. The detection rate reflects the proportion of actual hallucinations successfully caught by KGVS.	32
4.3	System detection performance per model showing TP, FP, FN, Precision, Recall, and F1 score.	33
4.4	Hallucination type breakdown per model showing generated, correctly detected, missed, and incorrectly categorised hallucinations. The “Wrong” column counts detections assigned an incorrect category.	34
A.1	Relationship types defined in the Neo4j knowledge graph schema.	43
B.1	Sample single-relationship questions from Dataset 1.	45
B.2	Sample two-relationship questions from Dataset 2.	45

Bibliography

- [1] Huang, L., Yu, W., Ma, W., Zhong, W., Feng, Z., Wang, H., Chen, Q., Peng, W., Feng, X., Qin, B., & Liu, T. (2025). A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *ACM Transactions on Information Systems*, 43(2), 1–56. <https://doi.org/10.1145/3703155>
- [2] Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y. J., Madotto, A., & Fung, P. (2023). Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12), 1–38. <https://doi.org/10.1145/3571730>
- [3] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, 9459–9474. <https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html>
- [4] Manakul, P., Liusie, A., & Gales, M. (2023). SelfCheckGPT: Zero-resource black-box hallucination detection for generative large language models. In H. Bouamor, J. Pino, & K. Bali (Eds.), *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing* (pp. 9004–9017). Association for Computational Linguistics. <https://aclanthology.org/2023.emnlp-main.557/>
- [5] Lu, X., Welleck, S., Hessel, J., Jiang, L., Qin, L., West, P., Ammanabrolu, P., & Choi, Y. (2023). Adaptable logical control for large language models. *arXiv preprint arXiv:2306.00092*. <https://arxiv.org/abs/2306.00092>
- [6] Sawczyn, A., Binkowski, J., Janiak, D., Gabrys, B., & Kajdanowicz, T. (2025). FactSelfCheck: Fact-level black-box hallucination detection for LLMs. *arXiv preprint arXiv:2501.07872*. <https://arxiv.org/abs/2501.07872>
- [7] Yang, K., & Klein, D. (2021). FUDGE: Controlled text generation with future discriminators. In K. Toutanova, A. Rumshisky, L. Zettlemoyer, D. Hakkani-Tur, I. Beltagy, S. Bethard, R. Cotterell, T. Chakraborty, & Y. Zhou (Eds.), *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies* (pp. 3511–3535). Association for Computational Linguistics. <https://aclanthology.org/2021.naacl-main.276/>

- [8] Kale, S., & Alfeo, A. L. (2025). Lie to me: Knowledge graphs for robust hallucination self-detection in LLMs. *arXiv preprint arXiv:2512.23547*. <https://arxiv.org/abs/2512.23547>
- [9] Wang, D., Peng, B., Liu, Z., Zhang, N., & Chen, J. (2022). TRACE back from the future: A probabilistic reasoning approach to controllable language generation. *arXiv preprint arXiv:2210.06647*. <https://arxiv.org/abs/2210.06647>
- [10] Zhang, H., Song, H., Li, S., Zhou, M., & Song, D. (2023). Tractable control for autoregressive language generation. *arXiv preprint arXiv:2304.07438*. <https://arxiv.org/abs/2304.07438>
- [11] Karatopak, E., Cekinel, R. F., & Yuret, D. (2024). Zero-knowledge LLM hallucination detection and mitigation through fine-grained cross-model consistency. *arXiv preprint arXiv:2405.16853*. <https://arxiv.org/abs/2405.16853>
- [12] Fang, X., Huang, Z., Tian, Z., Fang, M., Pan, Z., Fang, Q., Wen, Z., Pan, H., & Li, D. (2024). Zero-resource hallucination detection for text generation via graph-based contextual knowledge triples modeling. *arXiv preprint arXiv:2409.11283*. <https://arxiv.org/abs/2409.11283>
- [13] Lu, X., Welleck, S., Hessel, J., Jiang, L., Qin, L., West, P., Ammanabrolu, P., & Choi, Y. (2024). Sketch-guided constrained decoding for boosting blackbox large language models without logit access. *arXiv preprint arXiv:2401.09967*. <https://arxiv.org/abs/2401.09967>
- [14] Min, S., Krishna, K., Lyu, X., Lewis, M., Yih, W., Koh, P. W., Iyyer, M., Zettlemoyer, L., & Hajishirzi, H. (2023). FActScore: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation. In H. Bouamor, J. Pino, & K. Bali (Eds.), *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing* (pp. 12076–12100). Association for Computational Linguistics. <https://doi.org/10.18653/v1/2023.emnlp-main.741>
- [15] Hu, X., Ru, D., Qiu, L., Guo, Q., Zhang, T., Xu, Y., Luo, Y., Liu, P., Zhang, Y., & Zhang, Z. (2024). Knowledge-Centric Hallucination Detection. In Y. Al-Onaizan, M. Bansal, & Y. Chen (Eds.), *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing* (pp. 6953–6975). Association for Computational Linguistics. <https://doi.org/10.18653/v1/2024.emnlp-main.395>
- [16] Li, J., Cheng, X., Zhao, W. X., Nie, J.-Y., & Wen, J.-R. (2023). HaluEval: A Large-Scale Hallucination Evaluation Benchmark for Large Language Models. *CoRR*, abs/2305.11747. <https://doi.org/10.48550/arXiv.2305.11747>
- [17] Dong, Z., Tang, T., Li, J., Zhao, W. X., & Wen, J.-R. (2023). BAMBOO: A Comprehensive Benchmark for Evaluating Long Text Modeling Capacities of Large Language Models. *arXiv preprint*, abs/2309.13345. <https://arxiv.org/abs/2309.13345>

- [18] Lattimer, B. M., Chen, P., Zhang, X., & Yang, Y. (2023). Fast and Accurate Factual Inconsistency Detection Over Long Documents. *arXiv preprint*, abs/2310.13189. <https://arxiv.org/abs/2310.13189>
- [19] Chen, S., Zhao, Y., Zhang, J., Chern, I.-C., Gao, S., Liu, P., & He, J. (2023). FELM: Benchmarking Factuality Evaluation of Large Language Models. *arXiv preprint*, abs/2310.00741. <https://arxiv.org/abs/2310.00741>
- [20] Yang, S., Sun, R., & Wan, X. (2023). A New Benchmark and Reverse Validation Method for Passage-level Hallucination Detection. *arXiv preprint*, abs/2310.06498. <https://arxiv.org/abs/2310.06498>
- [21] Feng, H., Fan, Y., Liu, X., Lin, T.-E., Yao, Z., Wu, Y., Huang, F., Li, Y., & Ma, Q. (2023). Improving Factual Consistency of Text Summarization by Adversarially Decoupling Comprehension and Embellishment Abilities of LLMs. *CoRR*, abs/2310.19347. <https://doi.org/10.48550/arXiv.2310.19347>
- [22] Pan, L., Albalak, A., Wang, X., & Wang, W. Y. (2023). Logic-LM: Empowering large language models with symbolic solvers for faithful logical reasoning. *arXiv preprint*, abs/2305.12295. <https://arxiv.org/abs/2305.12295>
- [23] Sansford, H., Richardson, N., Petric Maretic, H., & Nait Saada, J. (2024). GraphEval: A Knowledge-Graph Based LLM Hallucination Evaluation Framework. In *Proceedings of the Workshop on Knowledge-infused Learning (KiL) at the 30th ACM KDD Conference*, Barcelona, Spain. arXiv preprint arXiv:2407.10793. <https://doi.org/10.48550/arXiv.2407.10793>
- [24] Yu, S., Lee, C., Lee, H., & Yoon, S. (2024). Controlled Text Generation for Black-box Language Models via Score-based Progressive Editor. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 14215–14237, Bangkok, Thailand. Association for Computational Linguistics. <https://doi.org/10.18653/v1/2024.acl-long.767>
- [25] Chern, I.-C., Chern, S., Chen, S., Yuan, W., Feng, K., Zhou, C., He, J., Neubig, G., & Liu, P. (2023). FacTool: Factuality Detection in Generative AI - A Tool Augmented Framework for Multi-Task and Multi-Domain Scenarios. *arXiv preprint*, abs/2307.13528. <https://arxiv.org/abs/2307.13528>
- [26] Zhang, B., & Soh, H. (2024). Extract, Define, Canonicalize: An LLM-based Framework for Knowledge Graph Construction. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 9820–9836, Miami, Florida, USA. Association for Computational Linguistics. <https://aclanthology.org/2024.emnlp-main.548/>
- [27] OpenAI. (2023). GPT-4 Technical Report. *arXiv preprint*, abs/2303.08774. <https://arxiv.org/abs/2303.08774>

- [28] Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., Rodriguez, A., Joulin, A., Grave, E., & Lample, G. (2023). LLaMA: Open and Efficient Foundation Language Models. *arXiv preprint*, abs/2302.13971. <https://arxiv.org/abs/2302.13971>
- [29] Anil, R., Borgeaud, S., Wu, Y., Alayrac, J.-B., Yu, J., Soricut, R., Schalkwyk, J., Dai, A. M., Hauth, A., Millican, K., Silver, D., Petrov, S., Johnson, M., Antonoglou, I., Schrittwieser, J., Glaese, A., Chen, J., Pitler, E., Lillicrap, T. P., Lazaridou, A., Firat, O., Molloy, J., Isard, M., Barham, P. R., Hennigan, T., Lee, B., Viola, F., Reynolds, M., Xu, Y., Doherty, R., Collins, E., Meyer, C., Rutherford, E., Moreira, E., Ayoub, K., Goel, M., Tucker, G., Piqueras, E., Krikun, M., Barr, I., Savinov, N., Danihelka, I., Roelofs, B., White, A., Andreassen, A., von Glehn, T., Yagati, L., Kazemi, M., Gonzalez, L., Khalman, M., Sygnowski, J., et al. (2023). Gemini: A Family of Highly Capable Multimodal Models. *CoRR*, abs/2312.11805. <https://arxiv.org/abs/2312.11805>
- [30] Zhao, W. X., Zhou, K., Li, J., Tang, T., Wang, X., Hou, Y., Min, Y., Zhang, B., Zhang, J., Dong, Z., et al. (2023). A Survey of Large Language Models. *arXiv preprint*, abs/2303.18223. <https://arxiv.org/abs/2303.18223>
- [31] GLM Team. (2024). ChatGLM: A Family of Large Language Models from GLM-130B to GLM-4 All Tools. *arXiv preprint*, abs/2406.12793. <https://arxiv.org/abs/2406.12793>
- [32] Mistral AI. (2024). Mistral Large. *Model Release Blog Post*. <https://mistral.ai/news/mistral-large>
- [33] Garcez, A. d., & Lamb, L. C. (2023). Neurosymbolic AI: The 3rd wave. *Artificial Intelligence Review*, 56(11), 12387–12406. <https://doi.org/10.1007/s10462-023-10448-w>